XEG: A GPU-parallel Algorithm for Efficient and Effective E-Graph Extraction

Yi-Hua Chung
University of Wisconsin-Madison
Madison, WI, USA
yihua.chung@wisc.edu

Hao-Hsiang Hsiao
Georgia Institute of Technology
Atlanta, GA, USA
thsiao@gatech.edu

Cheng-Hsiang Chiu
University of Wisconsin-Madison
Madison, WI, USA
chenghsiang.chiu@gmail.com

Boyang Zhang
University of Wisconsin-Madison
Madison, WI, USA
bzhang523@wisc.edu

Che Chang
University of Wisconsin-Madison
Madison, WI, USA
cchang289@wisc.edu

Joshua San Miguel
University of Wisconsin-Madison
Madison, WI, USA
jsanmiguel@wisc.edu

Tsung-Wei Huang
University of Wisconsin-Madison
Madison, WI, USA
tsung-wei.huang@wisc.edu

Abstract

Equality saturation is a promising compiler optimization approach that addresses the phase-ordering problem by compactly representing equivalent programs in an e-graph. However, extracting the optimal program from a saturated e-graph, known as *e-graph extraction*, is very time-consuming as the e-graph becomes dense and large. In this paper, we present XEG, a GPU-parallel e-graph extractor that achieves both fast runtime and high solution quality. Specifically, XEG introduces efficient algorithms to establish the e-graph and solution space on GPU and leverages ILP to extract a high-quality solution. Experimental results show that XEG (GPU-based) achieves a 243× average speedup over SmoothE (GPU-based) while improving solution quality by 7%. Compared with e-boost (CPU-based), XEG achieves a 7.73× average speedup while maintaining comparable solution quality. In addition, XEG can operate without ILP as a standalone extractor, achieving a 123× average speedup over egg (CPU-based) with comparable solution quality.

CCS Concepts: • **Computing methodologies** → **Massively parallel algorithms**.

Keywords: Compiler Optimization; GPU Algorithms

ACM Reference Format:
Yi-Hua Chung, Hao-Hsiang Hsiao, Cheng-Hsiang Chiu, Boyang Zhang, Che Chang, Joshua San Miguel, and Tsung-Wei Huang. 2027. XEG: A GPU-parallel Algorithm for Efficient and Effective E-Graph Extraction. In *Proceedings of the 32nd ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '27), April 11–15, 2027, Heraklion, Greece.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3845814.3850040



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '27, Heraklion, Greece
© 2027 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-3012-2/2027/04
https://doi.org/10.1145/3845814.3850040

1 Introduction

Term rewriting [16] is a widely used technique in compiler optimization [8, 37, 41, 48], in which the compiler sequentially applies rewrite rules to transform a program into a functionally equivalent but more efficient form. However, the effectiveness of such optimization heavily depends on the order in which these transformations are applied, a challenge known as the *phase ordering problem* [17, 44, 52]. To handle this problem, *equality saturation* [36, 49, 54, 55, 62] introduces a two-stage approach: *saturation* and *extraction*. During the saturation stage, all applicable rewrite rules are iteratively applied, and the resulting set of equivalent programs is compactly stored in a data structure called *equivalence graph* (*e-graph*). Once no further rewrite rules can be applied, the e-graph is considered saturated. In the extraction stage, a minimum-cost program is selected from the saturated e-graph according to a user-defined cost function.

However, due to its NP-hard nature, e-graph extraction is very time-consuming, and its runtime increases dramatically as the e-graph size grows [7, 47, 59, 61]. This poses a scalability challenge in applying e-graph to many large-scale optimization problems, such as logic synthesis in circuit optimization [7], which commonly generates e-graphs consisting of millions of nodes and edges. For example, as shown in Figure 1(a), extracting the optimal solution from a saturated logic synthesis e-graph of over 1M nodes can take more than 1 hour to finish [59]. Moreover, in such applications, e-graph extraction is typically invoked inside an optimization loop, where the circuit is iteratively optimized and rewritten, and

the per-extraction overhead can accumulate into a prohibitive overall runtime.

To address this runtime challenge, compiler researchers have introduced various solutions. To name just a few recent works: Greedy approaches [14, 40, 55] run very fast but often produce suboptimal results due to local decisions. To improve solution quality, Peggy and Tensat [47, 58] formulate extraction as an exact Integer Linear Programming (ILP), which guarantees optimal solutions but suffers from exponential runtime for large e-graphs. SmoothE [2] transforms e-graph extraction into a differentiable optimization problem with gradient-guided search, indirectly benefiting from GPU acceleration. However, SmoothE needs to store many intermediate gradient variables on GPU and can encounter out-of-memory (OOM) failures on large e-graphs. Recently, e-boost [59] proposes a CPU-parallel heuristic as a warm start for the ILP solver, enabling the ILP solver to obtain a high-quality solution faster. However, e-boost remains time-consuming as it requires the ILP solver to repeatedly avoid cycles during extraction. To address this issue, He et al. [24] preprocess all e-graph cycles as explicit ILP constraints. Yet, the preprocessing time can be very long (e.g., hours), as the number of cycles can grow exponentially in large e-graphs. As a consequence, existing e-graph extractors often trade solution quality for runtime and vice versa.

To overcome these challenges, we propose *XEG*, a GPU-parallel e-graph extractor. The key novelty of XEG lies in directly solving e-graph extraction via GPU parallelism, without resorting to proxy formulations such as differentiable optimization, or other approximations that compromise either speed or solution quality. As shown in Figure 1(b), XEG achieves both fast runtime and high solution quality simultaneously. We summarize our contributions as follows:

- We introduce a GPU-parallel algorithm that efficiently generates a high-quality initial solution and search space to facilitate the subsequent ILP solving.
- We introduce a GPU-parallel algorithm that progressively expands the search space without incurring cycles, enabling much faster ILP convergence.
- We introduce a bitmap data structure to efficiently support our GPU-parallel search-space construction algorithm. The bitmap avoids repeated graph traversals during e-node insertion, thereby significantly reducing construction time.

We evaluate the performance of XEG (GPU-based) on 87 e-graphs from diverse e-graph applications and compare its runtime and solution quality with three state-of-the-art e-graph extractors: egg [55] (CPU-based), SmoothE [2] (GPU-based), and e-boost [59] (CPU-based). Experimental results show that XEG achieves a 243× average speedup over SmoothE while improving solution quality by 7%. Compared with e-boost, XEG achieves a 7.73× average speedup while maintaining comparable solution quality. In addition, XEG can operate without ILP as a standalone extractor, achieving a 123× average speedup over egg with comparable solution quality.

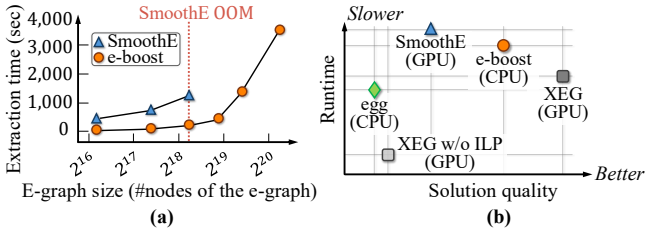


Figure 1. (a) State-of-the-art e-graph extractors remain time-consuming as the e-graph size increases [2, 59]. (b) Runtime vs solution quality trade-off of different e-graph extractors.

2 Background

Equality saturation translates an input program into an *e-graph*, which consists of equivalence classes (*e-classes*) and equivalence nodes (*e-nodes*) [35]. Each e-node encodes a program fragment as an abstract syntax tree (AST) node. Semantically equivalent e-nodes are grouped into the same e-class. After constructing the initial e-graph, the saturation stage iteratively applies user-defined rewrite rules to expand the e-graph with equivalent program fragments. This process repeats until no new e-nodes can be added, after which the e-graph is considered *saturated*, and the extraction stage begins. Figures 2(a)–(d) illustrate the saturation process of applying three rewrite rules to the input e-graph of $1 + (\sin(\alpha)\cos(\alpha)\sec^2(\alpha))^2 + \tan(\alpha)$.

E-graph extraction selects e-nodes from a saturated e-graph to construct a minimum-cost output program. To preserve functional equivalence, a valid extraction must satisfy [2, 24, 56]: (1) exactly one e-node is chosen from each root e-class; (2) each chosen e-node must select exactly one e-node from each of its child e-classes; and (3) the resulting program must be acyclic to avoid non-termination. Each e-node is assigned a user-defined local cost representing its performance impact, and a cost model aggregating these costs. The extraction problem is therefore to find a valid solution that minimizes the total cost.

The two widely used cost models are *tree model* and *DAG model* [2, 3, 19, 57, 59]. In the tree cost model, each selected e-node has a cumulative cost: its local cost plus the cumulative costs of the selected e-nodes in its child e-classes [3]. The total cost of the extracted program is the sum of the cumulative costs of the selected e-node(s) in the root e-class(es). Although this model enables polynomial-time extraction via recursive cumulative cost computation [59], it suffers from duplicate cost counting when an e-node has multiple parents, leading to overestimated results. For instance, in Figure 3(a), four e-nodes are selected and annotated with their local costs (top-left), with cumulative costs initialized to ∞ (top-right).

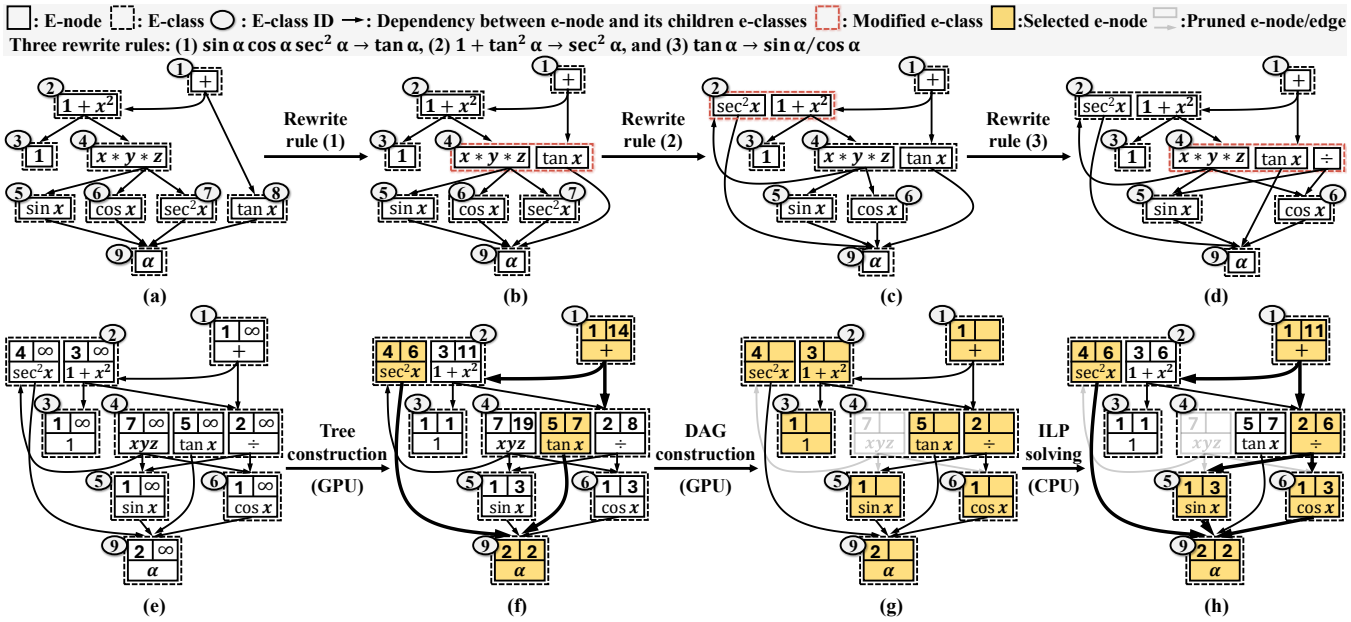


Figure 2. Example of equality saturation and e-graph extraction with XEG. Subfigures (a)–(d) illustrate equality saturation. Given the input program $1 + (\sin(\alpha)\cos(\alpha)\sec^2(\alpha))^2 + \tan(\alpha)$, the initial e-graph is shown in (a). Each e-node represents an operator, constant, or variable (e.g., $1 + x^2$ denotes a polynomial with two inputs, and $x * y * z$ represents a three-input multiplication). Subfigures (b)–(d) illustrate the saturation stage, where rewrite rules iteratively expand the e-graph by adding equivalent program fragments until no further terms can be added. Subfigures (e)–(h) illustrate the extraction stage with XEG, where the two top fields of each e-node represent its local cost (left) and cumulative cost (right). Starting from the saturated e-graph with the user-defined local costs, XEG first constructs a high-quality initial solution via tree construction, as detailed in Figure 4. It then progressively expands the search space through DAG construction by incorporating previously unused e-nodes while preserving acyclicity, as detailed in Figure 7. Finally, an ILP solver selects the minimum-cost solution under the DAG cost model, using the tree-construction result as a warm start and the acyclic search space produced by DAG construction. In this example, the extracted program is $\sec^2(\alpha) + \sin(\alpha)/\cos(\alpha)$, with a total cost of 11.

In Figure 3(b), recursively aggregating these costs yields a total cost of 6 ($2_{\text{local}} + 2_{\text{from } b} + 2_{\text{from } c}$), where d's local cost is counted twice because it is a child of both b and c.

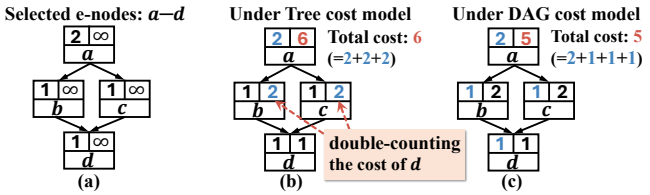


Figure 3. An example illustrating how the tree and DAG cost models compute total cost differently.

To address this issue, the more realistic DAG cost model defines the total cost as the sum of local costs over all selected e-nodes, counting each e-node exactly once. As shown in Figure 3(c), the shared e-node d is counted exactly once, resulting in a total cost of 5. This model is especially important for handling highly connected e-graphs, such as adder and c2670, where the tree-based model can mislead an extractor to choices that are suboptimal under the true DAG cost, by up to 30% [7]. However, adopting the DAG cost model significantly increases computational complexity and makes the problem NP-hard [47, 61], as e-node selections are no longer independent and must be optimized globally to account for shared descendants.

To overcome the above challenges, we aim to develop an e-graph extractor that can achieve both fast runtime and high solution quality under the DAG cost model.

3 XEG

We introduce XEG, a new extraction algorithm that focuses on improving the solution quality of existing extractors, while its algorithm naturally exposes high data parallelism. This characteristic makes XEG particularly suitable for GPU acceleration, although the algorithm can also be implemented on CPUs when GPUs are unavailable. As illustrated in Figure 2(e)–(h), XEG consists of three stages: *tree construction*, *DAG construction*, and *ILP solving*. The high-level idea behind tree construction is to address the high memory cost of prior work in generating a high-quality

initial solution. For example, e-boost [59] generates a high-quality initial solution under the DAG cost model to warm-start the ILP solver, but it requires additional memory to account for shared e-nodes without double counting their local costs. On the other hand, the key idea behind DAG construction is to reduce the long ILP runtime incurred by enforcing acyclicity, as a feasible solution cannot contain cycles. For example, He et al. [24] addresses the long ILP runtime by preprocessing cycles into explicit ILP constraints, but this preprocessing can be very time-consuming for large e-graphs with many cycles.

With these two ideas, the main contribution of XEG is a GPU-assisted parallel construction of a warm start and an acyclic restricted search space, followed by CPU-based ILP solving (i.e., heuristic extraction with exact solving over a restricted search space). Specifically, GPU-parallel tree construction generates a high-quality initial solution. Next, GPU-parallel DAG construction expands the search space by acyclically adding previously unselected e-nodes. Finally, ILP solves the extraction problem over the constructed search space under the DAG cost model.

3.1 Tree Construction

The state-of-the-art e-graph extractor e-boost [59] uses DAG-based extraction to warm-start its ILP solver. However, this approach incurs substantial memory overhead, as it must track all descendant e-nodes per e-node to avoid double counting, often leading to `OOM` failures on large e-graphs; it also requires repeated cycle detection during merges, serializing cost-dependent execution and limiting parallelism. Despite this complexity, the resulting solution quality is only slightly better than egg's greedy extraction under the tree cost model [59]. That being said, egg is architecturally limited to single-threaded execution, as it relies on sequential e-node updates in a bottom-up fashion, leaving significant data parallelism unexploited.

To address these challenges, we propose a GPU-parallel algorithm called *tree construction* that efficiently generates a high-quality initial solution to bootstrap the subsequent DAG construction and ILP solving stages. Specifically, our tree-construction algorithm does not rely on any memory-expensive or complex data structure like e-boost and egg. We relax the DAG cost to a tree model during tree construction and design a data-parallel layout where each e-node's cost is computed independently via a recursive sum on GPU. As we shall discuss in Section 4.2, our tree construction alone can serve as a standalone extractor when ILP is unavailable, achieving faster runtime than egg while providing comparable solution quality.

Our tree-construction algorithm consists of two steps: *bottom-up cost propagation* (BUCP) and *top-down tree construction* (TDTC). The goal of BUCP is to compute the cumulative cost of each e-node in a bottom-up fashion while identifying the minimum-cost e-node within each e-class. Specifically, we iteratively accumulate each e-node's local cost from its children's e-classes, where an e-class's cost is the minimum cost among its e-nodes. After all costs converge, TDTC selects the e-nodes that form the minimum-cost tree. To this end, we perform a parallel Breadth-First Search (BFS) [26, 34] by traversing all e-nodes at the same level simultaneously. We start from the e-nodes in the root e-classes that have the minimum cumulative costs. When an e-node is visited, we include it in the tree and continue the traversal through its child e-classes until we reach the leaves.

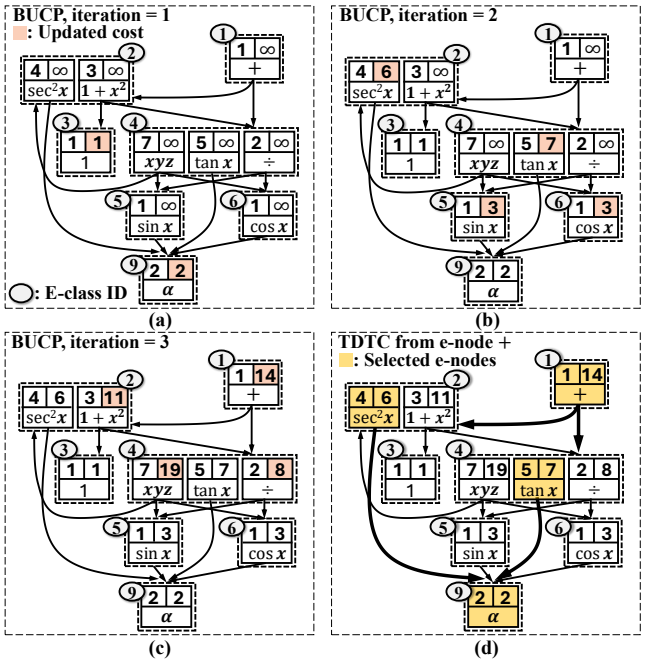


Figure 4. Illustration of tree-construction process for the example in Figure 2(e). Each e-node shows its local cost (top-left) and cumulative cost (top-right). (a)–(c) show step 1 (BUCP), where cumulative costs are updated until convergence. (d) shows step 2 (TDTC), where a BFS selects the lowest-cost e-node in each e-class to construct the minimum-cost tree.

Figure 4 shows the tree-construction process using the example of Figure 2(e). The local and cumulative costs of each e-node are shown at the top left and right, respectively, where ∞ denotes the initial value. Figures 4(a)–(c) illustrate the process of BUCP. In iteration 1, only the leaf e-nodes, α and 1, are updated (highlighted in pink) since they have no children, as in (a). In iteration 2, costs propagate upward, enabling interior e-nodes ($\sec^2 x$, $\tan x$, $\sin x$, and $\cos x$) to reduce their cumulative costs. After iteration 3, all e-nodes have been updated and no further changes occur, indicating that BUCP has converged, as in (c). We then move to TDTC and perform a BFS from the root e-class, where e-node "+" with the lowest cumulative cost is selected, as in (d). From

Algorithm 1: Tree construction

```
Input  : ende_local_cost: array mapping each e-node to its local cost
Input  : ende_ecls: array mapping each e-node to its e-class ID
Input  : ende_chd: array of child e-class IDs of all e-nodes
Input  : chd_id: [beg,end] indices into ende_chd per e-node
Input  : root_ecls: e-class IDs of extraction roots
Output : sel: selected e-nodes forming the minimum-cost tree
1  ecls_cost.init(∞); ende_cost.init(∞); best_ende.init(0)
2  ecls_lock.init(0); sel.init(false); upd = 1; frt = ∅
   /* Step 1. Bottom-up cost propagation (BUCP)          */
3  while upd do
4      upd = 0
5      parallel for each thread tid; Let ende = tid
6          ecls = ende_ecls[ende];
7          acc_cost = ende_local_cost[ende]
8          for i from chd_id[2*ende] to chd_id[2*ende+1]−1 do
9              acc_cost += ecls_cost[ende_chd[i]]
10         ende_cost[ende] = acc_cost
           /* Avoid unnecessary lock acquisition          */
11         cur_cost = atomicLoad(ecls_cost[ecls]);
12         if acc_cost < cur_cost then
               /* Atomically acquire the lock of this e-class */
13             while atomicCAS(ecls_lock[ecls], 0, 1) ≠ 0 do
14                 continue;
               /* Atomically update e-class cost          */
15             old_cost = atomicMin(ecls_cost[ecls], acc_cost);
               /* Update best e-node on a strict cost decrease */
16             if acc_cost < old_cost then
17                 best_ende[ecls] = ende; atomicOr(upd, 1);
18             atomicExch(ecls_lock[ecls], 0);
   /* Step 2. Top-down tree construction (TDTC)          */
19 foreach r ∈ root_ecls do
20     frt.push(best_ende[r]); sel[best_ende[r]] = true
21 while !frt.empty() do
22     nxt_frt = ∅; nxt_frt_size = 0
23     parallel for each thread tid; Let ende = frt[tid]
24         for i from chd_id[2*ende] to chd_id[2*ende+1]−1 do
25             chd_ecls = ende_chd[i]; nxt_ende = best_ende[chd_ecls]
26             old_sel = atomicCAS( sel[nxt_ende], false, true)
27             if old_sel == false then
28                 pos = atomicAdd(nxt_frt_size, 1); nxt_frt[pos] = nxt_ende
29     frt = nxt_frt
```

e-node +, the traversal proceeds to its child e-classes, 2 and 4, where the lowest-cost e-nodes $\sec^2 x$ and $\tan x$ are selected, respectively. Both lead to leaf e-class 9, where e-node α is selected. With α as a leaf, the BFS terminates and the tree is complete. The visited e-nodes (+, $\sec^2 x$, $\tan x$, and α) form the minimum-cost tree (yellow boxes with bold edges).

A key advantage of our tree-construction algorithm is its sufficient data parallelism in both steps, as the cumulative cost of each e-node can be computed independently in BUCP, and e-nodes at the same level can be processed in parallel in TDTC. To make the most of this parallelism, we introduce a GPU-parallel tree-construction algorithm (Algorithm 1). We use the classical compressed sparse row (CSR) data structure [4, 5, 10, 11, 25, 31] to store the input e-graph on GPU. The e-node local costs, e-class IDs, and root e-classes are stored in separate one-dimensional arrays. All e-nodes' child e-class IDs are concatenated into an adjacency list, with an adjacency pointer recording the start and end offsets of each e-node's child list.

Our algorithm consists of two steps: *BUCP* (lines 3–18) and *TDTC* (lines 19–29). In step 1, each GPU thread handles one e-node (line 5). It computes the cumulative cost by adding child e-class costs to the local cost (lines 6–10), and updates the e-class cost and best e-node if a smaller value is found (lines 11–18). Specifically, a thread first atomically reads the current e-class cost to avoid unnecessary lock acquisition (line 11). If its cumulative cost is strictly lower than the loaded e-class cost, it atomically acquires the lock of this e-class (lines 12–14). After acquiring the lock of this e-class, the thread atomically updates the e-class cost using its cumulative cost (line 15). If this update strictly lowers the e-class cost, the thread keeps this e-node as the best e-node of this e-class and sets the update flag (lines 16–17). It then releases the per-e-class lock (line 18). This repeats until no further updates occur. In step 2, we perform a BFS to construct the minimum-cost tree. Traversal begins with the best e-nodes of all root e-classes (lines 19–20) and proceeds level by level (lines 21–29). At each level, each GPU thread handles one e-node (line 23), selects the best e-node from each child e-class, and adds unvisited e-nodes to the next level (lines 24–28). The frontier is updated each iteration (line 29), and traversal terminates when all reachable e-nodes are visited, forming the minimum-cost tree.

Theorem 1. *If all e-nodes have non-negative local costs, Algorithm 1 outputs a globally minimum-cost tree by maintaining each e-class cost as the minimum cumulative cost among its e-nodes and updating its best e-node only when the e-class cost strictly decreases.*

Proof. We prove the theorem in terms of solution existence, acyclicity, and global optimality:

- **Existence:** Cumulative costs are monotonically non-increasing and lower-bounded by zero; the BUCP iteration converges to a finite value. Since the e-graph is initialized from a valid input program, at least one leaf-to-root path has finite cost, ensuring a valid solution can be constructed.
- **Acyclicity:** Assume the constructed tree contains a cycle $n_1 \to n_2 \to \cdots \to n_k \to n_1$, where n_i belongs to e-class E_i. Let $C(E_i)$ denote the cost of e-class E_i. Since n_i is the selected best e-node of E_i, $C(E_i)$ equals the cumulative cost of n_i. As each n_i has E_{i+1} as a child and all local and child costs are non-negative, we have $C(E_1) \geq C(E_2) \geq \cdots \geq C(E_k) \geq C(E_1)$. Thus, all e-class costs in the cycle are equal, i.e., $C(E_1) = C(E_2) = \cdots = C(E_k) = C$. Let t_i be the update that decreases $C(E_i)$ to C and selects n_i as its best e-node. For n_i to have cumulative cost C, its cycle child E_{i+1} must already have cost C. Hence, $t_{i+1} < t_i$ for

every i. Following the cycle gives $t_1 > t_2 > \cdots > t_k > t_1$, which is impossible. Thus, the constructed tree is acyclic.

- **Global optimality:** Let $cost(n)$ denote the cumulative cost of e-node n. Suppose a global minimum tree T^* selects a non-minimum-cost e-node n' for an e-class instead of a minimum cumulative-cost e-node n. As $cost(n) < cost(n')$, replacing the subtree rooted at n' with that rooted at n yields a valid tree with strictly lower cost, contradicting the optimality of T^*. Thus, a global minimum tree must select a minimum cumulative-cost e-node for each e-class. Note that if multiple e-nodes in an e-class have the same minimum cumulative cost, parallel execution may select any one of them. Although the selected e-node may depend on the update order, this affects only the extracted tree structure, not its optimal cost.

□

3.2 DAG Construction

Most existing works formulate e-graph extraction as an ILP problem to find the optimal solution [7, 8, 13, 15, 24, 51, 53, 58]. However, they often suffer from long runtime because the ILP solver needs to repeatedly enforce topological ordering for acyclic solutions [2, 24, 32, 59]. For example, e-boost requires over one hour to solve a large e-graph extraction problem with 1M e-nodes; although He et al. [24] improve ILP time by preprocessing cycles into explicit ILP constraints, the preprocessing time can be very long in large e-graphs that have many cycles, such as logic synthesis e-graphs [7].

To address these issues, we propose a GPU-parallel *DAG construction* algorithm that expands the tree from Algorithm 1 into a larger *acyclic* search space, enabling the ILP to efficiently find significantly better solutions while staying close to the original tree. Empirically, within practical time budgets, we found ILP can improve over the greedy solution with minimal structural deviation. Our algorithm comprises two steps: *search space expansion* and *search space filtering*. Starting from the initial tree, step 1 iteratively inserts unselected e-nodes, using topological levelization to ensure each insertion preserves acyclicity under the following conditions:

Condition 1. It has at least one selected parent or child e-node, ensuring connectivity of the search space.

Condition 2. The maximum level of its selected parent e-nodes does not exceed the minimum level of its selected child e-nodes ($l_{par_max} \leq l_{chd_min}$), preventing the introduction of a backward edge that could form a cycle.

Condition 3. No selected e-node is both a parent and a child of it, as this would create a cycle.

To increase opportunities for inserting unselected e-nodes, we alternate the levelization direction across iterations: a top-down (TD) levelization from roots to leaves and a bottom-up (BU) levelization from leaves to roots. This alternation is necessary because levelizing from different directions assigns different levels to selected e-nodes, which impacts the insertion of unselected e-nodes. As shown in Figure 5(a), given selected e-nodes 0–3, we attempt to insert unselected e-node 4. As e-node 4 has only one parent (e-node 3) and one child (e-node 2), both of which are selected, it satisfies conditions 1 and 3 above. However, as shown in (b), adopting TD levelization yields $l_{par_max} = 2$ and $l_{chd_min} = 1$, which violates condition 2. In contrast, under BU levelization, we obtain $l_{par_max} = 2$ and $l_{chd_min} = 2$, meeting condition 2 and thereby enabling e-node 4's insertion, as shown in (c).

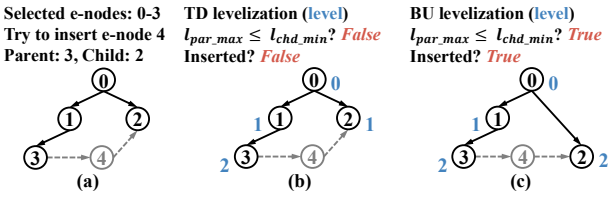


Figure 5. An example illustrating the impact of different levelization directions on e-node insertion.

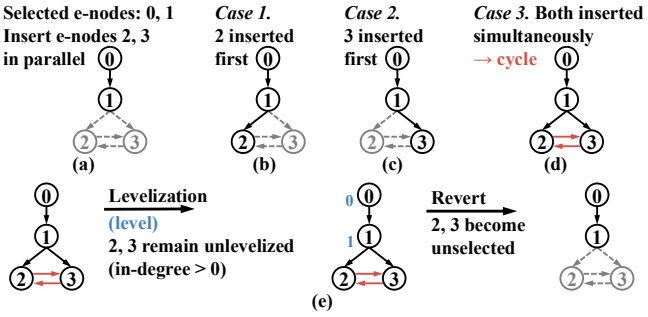


Figure 6. An example illustrating cycle creation and recovery during parallel e-node insertion.

A key advantage of our DAG construction is its sufficient data parallelism. Specifically, during levelization, e-nodes at the same level can be processed independently. However, parallel insertion may introduce cycles as insertion conditions can be verified using stale data. For example, in Figure 6(a), e-nodes 2 and 3 are mutually dependent and considered for parallel insertion. (1) If e-node 2 is inserted before e-node 3's conditions are verified, e-node 3 is rejected and no cycle forms (Figure 6(b)). (2) Conversely, if e-node 3 is inserted first, e-node 2 is rejected and no cycle forms (Figure 6(c)). (3) If e-nodes 2 and 3 are inserted simultaneously, a cycle occurs (Figure 6(d)). To fix this, we use levelization in the next iteration to identify and revert invalid insertions. Since e-graphs are connected, nodes with non-zero in-degrees after levelization are either part of a cycle or depend on cyclic upstream nodes [12, 28]. Hence, we track newly inserted e-nodes at each iteration. After levelization in the next iteration, we revert any e-node that still has non-zero in-degree back to

unselected. This removes cycles and preserves acyclic search space, thereby enabling parallel e-node insertion. As shown in Figure 6(e), e-nodes 2 and 3 are reverted to unselected. Note that some e-nodes may remain unselected after DAG construction to preserve acyclicity, so the resulting search space may not contain the global optimum. While this is a limitation of XEG, as we shall discuss in Section 4.6, DAG construction enables much faster ILP convergence with only 1% average solution-quality loss across all evaluated e-graphs.

Once no more e-nodes can be inserted, we proceed to step 2, search space filtering. Invalid and dangling e-nodes are removed, keeping only those that have a selected e-node in each of their child e-classes and that lie on a complete root-to-leaf path. First, a BFS from roots to leaves removes e-nodes whose child e-classes lack a selected e-node, then a BFS from leaves to roots removes remaining dangling e-nodes.

Figure 7 illustrates the DAG-construction process for Figure 2(f). We first flatten the e-graph into a directed graph by connecting each e-node to all e-nodes in its child e-classes and assigning each e-node a new ID, as shown in (a)–(b). For example, the e-node + (new ID 0) has two child e-classes, 2 and 4. e-class 2 contains e-nodes $\{sec^2x, 1+x^2\}$ (new IDs 1, 2), and e-class 4 contains $\{xyz, \tan(x), \div\}$ (new IDs 4–6). Thus, node 0 connects to nodes 1, 2, 4–6 in the flattened directed graph. Step 1 is shown in (c)–(e). Starting from the nodes selected by tree construction (yellow), we first perform TD levelization to assign levels (blue) to the selected nodes, as shown in (c). A level of -1 indicates an unselected or BFS-unreachable node. During insertion, we attempt to insert unselected nodes that meet the three insertion conditions. For instance, node 6 is successfully inserted because it meets all three conditions: it connects to a selected parent (node 0), has no selected children ($l_{par_max} = 1$ and $l_{chd_min} = \infty$), and no selected node is both its parent and child. After iteration 1, we assume nodes 2, 4, 6, 7, and 8 are inserted (pink) as all insertion conditions are met. In iteration 2, shown in (d), we switch to BU levelization. The selected nodes 0, 2, and 4 are assigned level -1 because they are unreachable due to a cycle between nodes 2 and 4 (red edges). Thus, nodes 2 and 4 are reverted to unselected as they are newly inserted from the previous iteration. In the following insertion, among the unselected nodes 2, 3, and 4, only nodes 2 and 4 are eligible, since node 3 has no selected parent or child. Assuming node 2 is inserted first, node 4 is rejected as node 2 becomes both its parent and child. Then, in the next iteration, we switch back to TD levelization, and node 3 is successfully inserted as shown in (e). Once no more nodes can be inserted, step 2 removes dangling nodes, yielding the final DAG in (f).

It can be observed that our DAG construction exhibits sufficient data parallelism, as levelization allows all e-nodes at the same level to run independently. Likewise, data parallelism exists in e-node insertion as we can insert unselected e-nodes in parallel, since any cycles introduced can be corrected in the next iteration. To fully exploit these parallelisms,

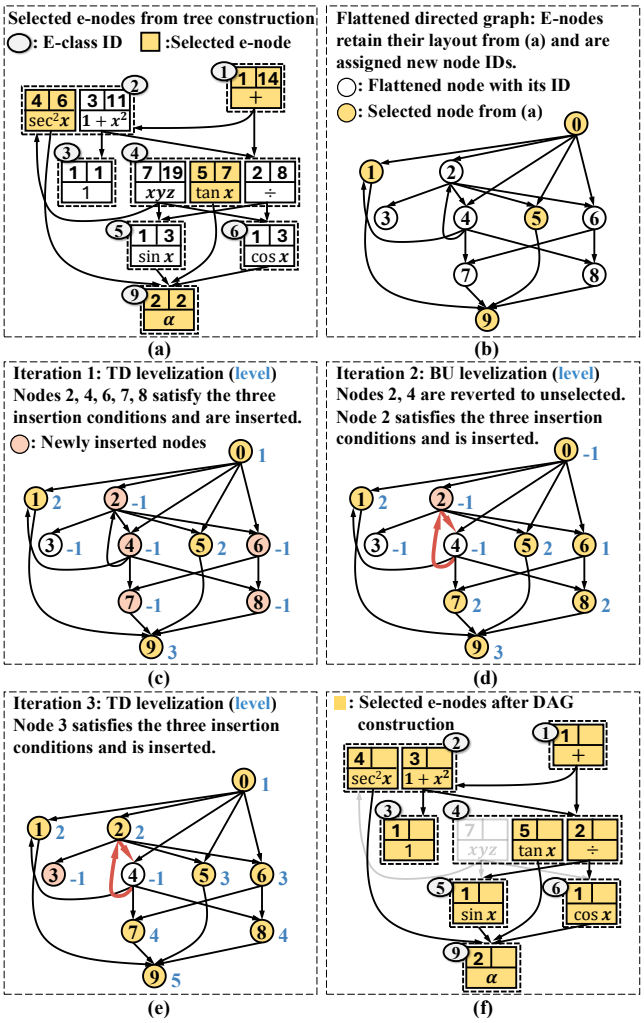


Figure 7. Illustration of DAG-construction process for the example in Figure 2(f). (a)–(b) show the initial tree solution and its flattened graph. (c)–(e) show iterative search space expansion with levelization and insertion. (f) shows the final acyclic search space.

we introduce a GPU-parallel DAG-construction algorithm (Algorithm 2), consisting of two steps: *search space expansion* (lines 34–41) and *search space filtering* (lines 42–44). The input e-graph is stored on GPU in CSR format, including arrays for selected e-nodes, parent/child adjacency lists, and root/leaf e-node IDs. In step 1, the algorithm starts with TD levelization and repeatedly alternates between insertion and levelization (BU and TD directions) until no new e-nodes are inserted (lines 34–41). The *BFS_levelization* function assigns levels to selected e-nodes via BFS (lines 1–21). We first count in-degrees in parallel (lines 2–6). E-nodes with zero in-degree are processed first, followed by level-by-level traversal (lines 7–14). To prevent cycles, any newly selected e-node with non-zero in-degree after levelization is reverted

Algorithm 2: DAG construction

```
Input  : sel: initial selected e-nodes (from tree construction)
Input  : adj/adj^-1: adjacency lists (child/parent)
Input  : roots/leaves: array of all root/leaves e-nodes
Output : sel: selected e-nodes forming the acyclic search space
1  Function BFS_levelization(is_td, sel, lvl, new, u0):
2      deg.init(0); lvl.init(-1); frt = ∅; nxt_frt = ∅; cur_lvl = 0
3      nbr = (is_td ? adj : adj^-1)
4      parallel for each thread tid; Let u = tid // Count in-degrees
5          if sel[u] then
6              foreach v ∈ nbr[u] do if sel[v] then atomicAdd(deg[v], 1)
7      parallel for each thread tid; Let u = tid    // Init frontier
8          if sel[u] ∧ deg[u] == 0 then frt.push(u)
9      while !frt.empty() do            // Levelize selected nodes
10         parallel for each thread tid; Let u = frt[tid]
11             lvl[u] = cur_lvl; foreach v ∈ nbr[u] do
12                 if sel[v] then
13                     d = atomicSub(deg[v], 1); if d == 1 then nxt_frt.push(v)
14         frt = nxt_frt; nxt_frt = ∅; cur_lvl++
       /* Revert invalid insertions after levelization    */
15     parallel for each thread tid; Let u = tid
16         if sel[u] ∧ deg[u] > 0 ∧ new[u] then
17             sel[u] = false; atomicSub(u0, 1)
18         new[u] = false          // Reset insertion flag
19     if !is_td then // Reverse BU levels to match TD ordering
20         parallel for each thread tid; Let u = tid
21             if sel[u] then lvl[u] = cur_lvl - lvl[u] - 1
22 Function ende_insertion(sel, lvl, new, u0):
23     parallel for each thread tid; Let u = tid
24         if sel[u] then return
           /* Check insertion Conditions 1-3                  */
25         P_sel = {p | p ∈ adj^-1[u] ∧ lvl[p] ≠ -1}
26         C_sel = {c | c ∈ adj[u] ∧ lvl[c] ≠ -1}
27         if (P_sel == ∅ ∧ C_sel == ∅) then return
28         l_par_max = max({lvl[p] | p ∈ P_sel} ∪ {-1})
29         l_chd_min = min({lvl[c] | c ∈ C_sel} ∪ {∞})
30         t_lvl = P_sel ≠ ∅ ? l_par_max + 1 : l_chd_min - 1
31         if (l_par_max > l_chd_min) ∨ (t_lvl < 0) then return
32         cycle = ((P_sel ∩ C_sel) ≠ ∅); if cycle then return
33         sel[u] = true; lvl[u] = t_lvl; new[u] = true; atomicAdd(u0, 1)
   /* Step 1. Search space expansion                       */
34 new.init(false)        // Initialize insertion flags
35 BFS_levelization(true, sel, lvl, new, u0); u0 = 1
36 while u0 > 0 do
37     u0 = 0              // Reset insertion count
       /* TD insertion; BU levelization                   */
38     ende_insertion(sel, lvl, new, u0)
39     BFS_levelization(false, sel, lvl, new, u0)
       /* BU insertion; TD levelization                   */
40     ende_insertion(sel, lvl, new, u0)
41     BFS_levelization(true, sel, lvl, new, u0)
   /* Step 2. Search space filtering                       */
42 rt = BFS(roots, adj, sel); lf = BFS(leaves, adj^-1, sel)
43 parallel for each thread tid; Let u = tid
44     if (lvl[u] == -1) ∨ !(rt[u] ∧ lf[u]) then sel[u] = false
```

to unselected (lines 15–18). For BU levelization, the assigned levels are reversed (lines 19–21). The *ende_insertion* function inserts unselected e-nodes while preserving acyclicity. Each thread processes one e-node, checking conditions 1–3 (lines 25–27, 28–31, and 32). If all conditions hold, the e-node is inserted and marked as selected (line 33). To guarantee the termination of step 1, we use an insertion count for each iteration (line 37). The insertion count is incremented when an e-node is inserted by *ende_insertion* (line 33) and decremented if that insertion is reverted by the following *BFS_levelization* (line 17). Since every *ende_insertion* is followed by a *BFS_levelization* (lines 38–41), any newly inserted e-node that introduces a cycle is reverted and removed from the insertion count. Once no insertion remains after the levelization, the insertion count becomes zero and the while loop terminates. After step 1, BFS is performed from roots and leaves to remove invalid and dangling e-nodes, forming the final DAG (lines 42–44).

3.3 GPU Kernel Optimization for DAG Construction

DAG construction runtime is dominated by e-node insertion, as verifying the insertion conditions requires traversing each unselected e-node's parent and child adjacency lists (lines 25–32 in Algorithm 2). In particular, checking condition 3 involves a nested traversal over both parent and child lists (line 32 in Algorithm 2). Although insertions are parallelized, each individual insertion is handled by a single thread. As a result, when an unselected e-node has a high vertex degree, the responsible thread incurs a heavy computational workload, significantly increasing DAG construction time.

To overcome this runtime challenge, we propose an optimized e-node insertion kernel with two enhancements. First, instead of assigning one thread per insertion, we assign a group of threads to cooperatively process each e-node, parallelizing the heavy workload of adjacency-list traversal. Second, we introduce a bitmap structure that replaces the costly nested loop for checking condition 3 with two sequential traversals over the parent and child adjacency lists. We allocate two blocks of shared memory, one for the parent bitmap and one for the child bitmap, which are shared by all threads mapped to the same e-node. Given a bitmap of size M, each parent or child e-node ID is mapped to the corresponding bitmap at index $index = ID\%M$ during the traversals. After both traversals, we compute the bitwise intersection of the parent and child bitmaps. If the intersection is non-zero, condition 3 is violated, and the e-node insertion is rejected. For example, as shown in Figure 8, consider e-node 0 with $M = 8$. Its parent e-nodes $\{6, 8\}$ are mapped to bits 6 and 0 in the parent bitmap, while its child e-nodes $\{1, 6, 7\}$ are mapped to bits 1, 6, and 7 in the child bitmap. The non-zero bitwise intersection at bit 6 indicates a cycle, and the insertion is therefore rejected. In our implementation, each insertion is handled by a group of 64 threads with $M = 512$ (tunable), which achieves the best runtime results for our benchmarks.

We introduce our optimized e-node insertion in Algorithm 3. Each insertion is handled by N_t threads (line 2). Threads first traverse parent and child adjacency lists in

Algorithm 3: Optimized *ende_insertion* with bitmap

```
Input  : N_t: number of threads assigned to process one e-node
Input  : adj/adj^-1: adjacency lists (child/parent)
Output : sel: selected e-nodes forming the acyclic search space
Memory : B_par, B_chd: M-bit bitmaps shared by N_t (init to 0)
1  Function ende_insertion_opt(sel, lvl, new, u0):
2    parallel for each thread tid; Let u = tid / N_t; l_tid = tid % N_t
3      if sel[u] then return
4      l_par_loc = -1; l_chd_loc = ∞; has_cycle = false
       /* Scan parents and atomically update bitmap      */
5      for i = l_tid to |adj^-1[u]| - 1 step N_t do
6        p = adj^-1[u][i]
7        if lvl[p] ≠ -1 then
8          l_par_loc = max(l_par_loc, lvl[p])
9          atomicSetBit(B_par, p % M)
       /* Scan children and atomically update bitmap     */
10     for i = l_tid to |adj[u]| - 1 step N_t do
11       c = adj[u][i]
12       if lvl[c] ≠ -1 then
13         l_chd_loc = min(l_chd_loc, lvl[c])
14         atomicSetBit(B_chd, c % M)
15     Barrier();      // Synchronize bitmap construction
16     l_par_max = ReduceMaxSync(l_par_loc)
17     l_chd_min = ReduceMinSync(l_chd_loc)
       /* Check Conditions 1-2 using level bounds        */
18     if l_par_max == -1 ∧ l_chd_min == ∞ then return
19     t_lvl = (l_par_max ≠ -1 ? l_par_max + 1 : l_chd_min - 1)
20     if (l_par_max > l_chd_min) ∨ (t_lvl < 0) then return
       /* Check Condition 3 via bitmap intersection      */
21     for k = l_tid to M - 1 step N_t do
22       if B_par[k] & B_chd[k] then has_cycle = true;
23     if Any(has_cycle) then return
24     if l_tid == 0 then // Commit insertion by leader thread
25       sel[u] = true; lvl[u] = t_lvl; new[u] = true; atomicAdd(u0, 1)
```

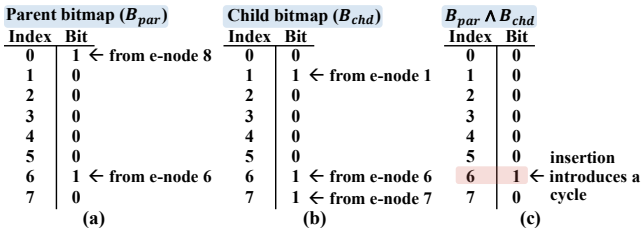


Figure 8. An example of cycle detection via bitmap intersection with $M = 8$. For e-node 0, parent e-node IDs $\{6, 8\}$ and child e-node IDs $\{1, 6, 7\}$ are mapped into bitmaps. The bitwise intersection at index 6 indicates a cycle between e-node 0 and 6.

parallel with stride N_t (lines 5–9 and 10–14). Then, they update the local maximum parent level and minimum child level (lines 8 and 13) by atomically setting the corresponding bitmap bits using `atomicSetBit`, implemented with `atomicOr` (lines 9 and 14). After synchronization (line 15), the N_t threads in each group perform reductions to compute l_{par_max} and l_{chd_min} (lines 16–17), then verify conditions 1–2 using these level bounds and condition 3 via bitmap intersection (lines 18–23). If all conditions hold, the e-node is selected (lines 24–25).

3.4 ILP Solver

We formulate the extraction problem as an ILP based on the constructed DAG by Algorithm 2. Since the search space is acyclic, we eliminate the expensive enforcement of topological constraints required by existing work [8, 9, 13, 15, 24, 33, 45, 46, 51, 53, 54, 59]. Our formulation is presented below:

$$\min_{s_i, A_j \in \{0,1\}} \quad \sum_{i \in \mathcal{N}_{dag}} \text{cost}(i) \cdot s_i \tag{1a}$$

$$\text{s.t.} \quad \sum_{i \in \mathcal{N}_j} s_i = A_j, \quad \forall j \in \mathcal{C}_{dag} \tag{1b}$$

$$s_i \le A_k, \quad \forall i \in \mathcal{N}_{dag}, \forall k \in \text{children}(i) \tag{1c}$$

$$A_r = 1, \quad \forall r \in \text{Roots} \tag{1d}$$

where $\mathcal{N}_{dag}$ and $\mathcal{C}_{dag}$ denote the sets of e-nodes and e-classes in the constructed DAG. Binary variables s_i and A_j indicate whether e-node i and e-class j are selected, respectively. The objective (1a) minimizes the total extraction cost under the DAG cost model. Constraint (1b) ensures that for each selected e-class, exactly one e-node is selected from its node set ($\mathcal{N}_j$). Constraint (1c) enforces dependencies: if an e-node i is selected, all its child e-classes must be selected. Constraint (1d) requires all root e-classes to be selected. To accelerate ILP solving, we use the solution from tree construction (Algorithm 1) as an initial feasible solution for the ILP solver.

4 Experimental Results

We evaluated the performance of XEG on 87 e-graphs from nine different benchmarks that cover diverse e-graph applications, including BoolE [60], ESyn [7], E-morphic [6], Diospyros [53], IMpress [51], Rover [15], Tensat [58], Flexc [56], and Maxsat [2]. Table 1 summarizes the benchmark statistics. We implemented XEG in CUDA and C++, and compiled it using nvcc v12.3 with `-O3` and `-std=c++20` enabled. All experiments were conducted on a 64-bit Linux machine equipped with an Intel Core i5-13500 processor (32 threads, 4.8 GHz), 128 GB of RAM, and an NVIDIA RTX A4000 GPU with 16 GB global memory. All results are averaged over 10 runs.

4.1 Baselines and Evaluation Metrics

We consider three state-of-the-art e-graph extractors, *egg* [55] (CPU-based), *SmoothE* [2] (GPU-based), and *e-boost* [59] (CPU-based) as our baselines. For egg, we adopt the default bottom-up mode as it produces the best result. For e-boost, we use the same parameter described in [59]. For XEG, we use the block-level implementation with bitmap acceleration for the DAG-construction stage, as it yields the best result. We use CP-SAT [29] as the ILP solver for both e-boost and XEG, as it outperforms other ILP solvers [21, 27, 59], including a GPU-based solver discussed in Appendix A. In all experiments, the CP-SAT timeout is set at 360 seconds where all baselines can converge.

Table 1. Performance comparison between XEG and state-of-the-art e-graph extractors, egg [55], SmoothE [2], and e-boost [59] on nine benchmarks. For each benchmark, #G denotes the number of e-graphs, #e-classes/#e-nodes denote the size range across e-graphs, and $\bar{d}^{NC}/\bar{d}^{NN}$ denote the average output degrees from e-nodes to e-classes/e-nodes. Avg. runtime ratio is the average speedup of XEG over the baselines across e-graphs in the benchmark. Avg. normalized cost is the average ratio of the baseline's final cost to XEG's. The **bold** number in parentheses indicates the number of e-graphs that fail due to OOM. Detailed per-e-graph statistics and results are available in Appendix B.

Benchmark	Statistics					Avg. runtime ratio				Avg. normalized cost			
	#G	#e-classes	#e-nodes	$\bar{d}^{NC}$	$\bar{d}^{NN}$	egg	SmoothE	e-boost	XEG	egg	SmoothE	e-boost	XEG
BoolE	16	[4.2K, 619K]	[10.5K, 1.2M]	1.81	4.24	0.097	≥ 38.28 (**9**)	≥ 5.28 (**4**)	1.00	1.23	≥ 1.22 (**9**)	≥ 1.12 (**4**)	1.00
ESyn	14	[3.3K, 15.1K]	[8.5K, 32.0K]	1.76	8.92	0.006	13.40	19.44	1.00	1.12	1.09	0.99	1.00
E-morphic	7	[13.4K, 146K]	[20.4K, 190K]	1.67	2.81	0.008	5.82	7.78	1.00	1.04	1.03	1.00	1.00
IMpress	3	[9.1K, 90.0K]	[10.2K, 102K]	2.00	2.10	0.159	157	8.23	1.00	1.60	1.12	0.92	1.00
Tensat	5	[1.4K, 34.8K]	[2.7K, 57.8K]	2.39	5.25	0.083	≥ 794 (**1**)	4.38	1.00	1.11	≥ 1.00 (**1**)	1.02	1.00
Flexc	14	[1.7K, 4.8K]	[6.8K, 19.8K]	1.87	49.91	0.174	400	4.53	1.00	1.01	1.02	1.00	1.00
Rover	9	[349, 2.9K]	[1.8K, 17K]	9.14	196	0.026	38.63	2.19	1.00	1.01	1.15	1.00	1.00
Diospyros	13	[17.0, 9.6K]	[91.0, 219K]	2.13	66.22	0.219	≥ 777 (**3**)	8.86	1.00	1.00	≥ 1.00 (**3**)	1.00	1.00
Maxsat	6	[1.4K, 3.8K]	[1.5K, 3.9K]	1.84	1.90	0.025	18.29	1.12	1.00	1.00	1.00	1.00	1.00
Overall Avg.			-			0.095	≥ 243	≥ 7.73	1.00	1.09	≥ 1.07	≥ 1.01	1.00

We evaluate the performance in terms of runtime and solution quality. For runtime, we define *runtime ratio* as the baseline runtime over XEG (i.e., a larger ratio means XEG achieves a higher speedup). For baselines that fail to produce a valid extraction due to OOM errors, we exclude their runtime from the average runtime ratio to avoid skewing the results. For solution quality, we define the *normalized cost* as the final objective cost achieved by a baseline over XEG under the DAG cost model (i.e., smaller is better). Similarly, for baselines that fail to produce a valid solution due to OOM, we use the largest cost obtained by other baselines on the same e-graph to keep the average normalized cost finite.

4.2 Overall Performance Comparison

Table 1 compares the performance of XEG against the three baselines, egg, SmoothE, and e-boost across all evaluated e-graphs (detailed version in Appendix B). In terms of runtime, XEG consistently outperforms both SmoothE and e-boost across all benchmarks, with speedups of 5.82×–777× over SmoothE and 1.12×–19.44× over e-boost. Overall, XEG achieves an average speedup of 243× over SmoothE and 7.73× over e-boost across all evaluated e-graphs. We attribute this runtime advantage to: (1) XEG's tree construction, which helps the ILP solver start with a high-quality initial feasible solution; (2) XEG's DAG construction, which starts from the initial solution and progressively expands the search space by adding previously unused e-nodes that do not introduce cycles, enabling the ILP solver to converge significantly faster. In contrast, egg is 10.53× faster than XEG on average, because its greedy extraction makes local decisions under the tree cost model, whereas XEG performs a more global optimization under the DAG cost model.

In terms of solution quality, XEG achieves the best overall normalized cost of 1.00 across all evaluated e-graphs, whereas egg is 1.09, SmoothE is ≥1.07, and e-boost is ≥1.01. Note that for SmoothE and e-boost, these values are best-case lower bounds, as we exclude e-graphs where they fail to produce a solution. We observe the largest difference between XEG and egg in the IMpress benchmark, where egg has a high cost of 1.60. This is because egg's greedy extraction algorithm often gets trapped in local minima, whereas XEG's DAG construction exposes a more global and structural view of the e-graph, enabling the ILP to avoid such local minima. The largest difference between XEG and SmoothE appears in BoolE, where SmoothE is no better than 1.22, primarily due to the GPU OOM problem on large e-graphs. Likewise, for e-boost, its quality on BoolE is no better than 1.12 because it fails to complete the extraction on large e-graphs due to CPU OOM errors caused by its warm-start initialization. In summary, we attribute the low cost of XEG to its DAG construction, which provides the ILP solver with a high-quality acyclic search space to explore. The acyclic property enables the solver to converge more quickly, allowing it to evaluate more feasible solutions within the same runtime budget and ultimately reach lower objective costs.

Figure 9 compares the convergence speed to a high-quality solution between XEG and baselines. XEG consistently outperforms other baselines by converging faster to lower cost values. For example, on c7552, b12, sin, and mul32 ((a)–(d)), XEG converges in less than a minute (< 27s), whereas e-boost can take 11.38×–62.45× longer to reach a cost comparable to XEG. On the other hand, SmoothE takes a longer time to converge, yet its solution quality is far from XEG and e-boost. For larger e-graphs such as mul112 and mul96map ((e)–(f)), only egg and XEG can finish, while SmoothE and e-boost incur OOM.

Figure 9 does not show the curve for egg because its greedy algorithm finishes very fast. However, its solution quality is not competitive, especially for large e-graphs (e.g., 35% and 20% worse than XEG on mul112 and mul96map, respectively).

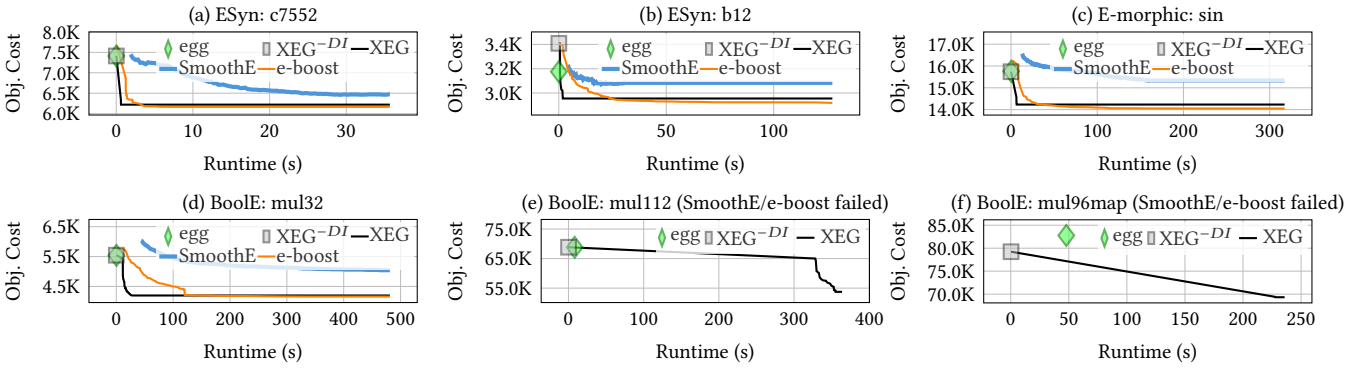


Figure 9. Comparison of the convergence speed to a high-quality solution for egg, SmoothE, e-boost, XEG, and XEG^{-DI}. Overall, XEG outperforms other baselines by converging faster to low objective cost (Obj. Cost) values, including large benchmarks (mul112 and mul96map) where SmoothE and e-boost fail to produce valid solutions due to `OOM` errors.

That said, when the runtime overhead of ILP becomes prohibitive, our GPU-parallel tree construction algorithm (XEG^{-DI}) can serve as a standalone extractor, delivering high solution quality at substantially lower runtime. As shown in Table 2, XEG^{-DI} achieves a 123× speedup over egg while yielding comparable solution quality. We attribute this improvement to our GPU-parallel data structure, which overcomes the limitation of sequential update in egg.

Table 2. Comparison of egg and our tree construction (XEG^{-DI}) alone in terms of runtime and solution quality for each benchmark.

Benchmark	Avg. runtime ratio		Avg. normalized cost	
	egg	XEG^{-DI}	egg	XEG^{-DI}
BoolE	389.00	1.00	1.02	1.00
ESyn	9.79	1.00	1.01	1.00
E-morphic	149.00	1.00	1.00	1.00
IMpress	85.69	1.00	1.00	1.00
Tensat	35.60	1.00	1.00	1.00
Flexc	17.29	1.00	1.01	1.00
Rover	48.61	1.00	1.00	1.00
Diospyros	163.00	1.00	1.00	1.00
Maxsat	4.73	1.00	1.00	1.00
Overall Avg.	**123.00**	**1.00**	**1.01**	**1.00**

4.3 Effectiveness of Tree Construction

In this section, we study the impact of our tree construction on overall runtime and solution quality by comparing XEG with a variant that does not use it to initialize the ILP solver. We denote this variant as XEG^{-T}, which instead initializes the ILP by randomly selecting one e-node from each e-class. Note that although e-boost adopts a similar warm-start strategy, we do not include it in this comparison as its initialization algorithm modifies the e-graph (e.g., pruning e-nodes), which in turn alters the resulting ILP structure and makes a fair comparison difficult. Similarly, since egg and SmoothE are non-ILP-based, we do not compare with them.

As shown in Table 3, XEG achieves an average speedup of 1.52× over XEG^{-T}. The largest average speedup is 3.01× over XEG^{-T} on the ESyn benchmark. This is because ESyn exhibits large cost differences among e-nodes within the same e-class, making XEG^{-T} likely to pick expensive e-nodes for ILP solver to start from, whereas XEG selects low-cost e-nodes through tree construction and begins with a much better initial solution. Across all benchmarks, XEG consistently outperforms XEG^{-T} by 1.01× to 3.01×. Regarding solution quality, XEG and XEG^{-T} achieve comparable normalized costs on most benchmarks. On BoolE, XEG performs even better than XEG^{-T} (1.00 vs 1.09). This improvement arises because tree construction provides a stronger initial solution, enabling DAG construction to build a better search space and allowing XEG to converge to a better cost.

Table 3. Effectiveness of tree construction and DAG construction on the overall runtime and normalized cost for each benchmark. XEG^{-T} and XEG^{-D} denote variants without tree construction and DAG construction, respectively.

Benchmark	Avg. runtime ratio			Avg. normalized cost		
	XEG^{-T}	XEG^{-D}	XEG	XEG^{-T}	XEG^{-D}	XEG
BoolE	1.11	6.16	1.00	1.09	1.13	1.00
ESyn	3.01	99.12	1.00	1.01	0.99	1.00
E-morphic	1.01	6.07	1.00	1.00	1.00	1.00
IMpress	1.08	16.06	1.00	1.00	0.92	1.00
Tensat	1.09	19.26	1.00	1.00	0.93	1.00
Flexc	1.60	13.75	1.00	1.00	1.00	1.00
Rover	1.36	20.06	1.00	1.00	0.98	1.00
Diospyros	1.19	17.40	1.00	1.00	1.00	1.00
Maxsat	1.01	0.98	1.00	1.00	1.00	1.00
Overall Avg.	**1.52**	**26.19**	**1.00**	**1.02**	**1.01**	**1.00**

Although particularly suitable for GPU parallelism, XEG's tree construction can also be implemented on CPUs when GPUs are unavailable. To demonstrate this applicability, we implemented a CPU reference that uses OpenMP `parallel-for` to parallelize our tree construction algorithm by

(1) computing e-node costs in parallel (lines 5–18 in Algorithm 1) and (2) processing frontier e-nodes in parallel (lines 23–28 in Algorithm 1). Atomic operations are used for concurrent updates to e-class costs, update flags, selection states, and frontier positions (lines 16, 17, and 26–28 in Algorithm 1). We believe this OpenMP implementation is reasonably well-engineered because the shared e-class costs must be updated immediately during bottom-up cost propagation such that newly updated e-class costs can be consumed by other e-nodes within the same iteration.

As shown in Table 4, relative to the 1-thread CPU implementation, the 8-, 16-, and 32-thread CPU implementations achieve geometric-mean speedups of 2.89×, 3.44×, and 3.69×, respectively, while the GPU implementation further achieves an 8.24× speedup. Note that this CPU reference achieves the same solution quality as the GPU implementation because it implements the same algorithm. Also, the reported runtime here is intended to help readers understand the potential performance gains from the overall system perspective of our setting, since the total runtime largely relies on the chosen ILP solver.

Table 4. Runtime of our tree-construction algorithm on CPUs and GPU. Runtimes are in milliseconds (ms); values in parentheses denote the percentage (%) of the total runtime. The bottom row reports the geomean speedup over the 1-thread CPU.

Benchmark	CPU				GPU
	1	8	16	32	
BoolE	212 (0.39)	59.40 (0.11)	46.07 (0.08)	37.46 (0.07)	13.9 (0.02)
ESyn	20.64 (0.12)	3.19 (0.02)	2.95 (0.02)	2.88 (0.02)	2.92 (0.03)
E-morphic	730 (1.33)	70.39 (0.13)	35.98 (0.07)	30.36 (0.06)	19.72 (0.04)
IMpress	12.13 (4.86)	4.00 (1.60)	3.17 (1.27)	2.95 (1.18)	0.65 (0.26)
Tensat	7.59 (3.06)	2.98 (1.20)	2.44 (0.98)	2.23 (0.90)	1.39 (0.52)
Flexc	2.67 (3.51)	1.22 (1.61)	1.09 (1.43)	1.09 (1.43)	0.48 (1.11)
Rover	2.19 (0.21)	1.22 (0.12)	1.08 (0.10)	1.09 (0.10)	0.43 (0.08)
Diospyros	1.55 (1.40)	1.19 (1.07)	1.12 (1.02)	1.08 (0.98)	0.23 (1.71)
Maxsat	0.78 (1.86)	0.58 (1.38)	0.57 (1.35)	0.54 (1.28)	0.28 (0.58)
Speedup	**1.00**	**2.89**	**3.44**	**3.69**	**8.24**

4.4 Effectiveness of DAG Construction

In this section, we study the impact of our DAG construction on the overall runtime and solution quality by comparing XEG with a variant that disables it, denoted as XEG^{-D}. As shown in Table 3, XEG achieves an average speedup of 26.19× over XEG^{-D}. The largest average speedup we observe is 99.12× over XEG^{-D} on ESyn, since it is larger and contains more cycles than other benchmarks. In contrast, for small benchmarks like Maxsat, DAG construction does not provide much benefit, as the e-graph size is small and the tree construction is already good enough for ILP to find a decent solution. Yet, for most benchmarks, DAG construction brings significant runtime advantages as it produces an acyclic search space for the ILP solver to converge faster. For example, as shown in Figure 10, on e-graphs c7552, b12, sin, and mul32 ((a)–(d)), DAG construction allows XEG to converge 207×, 39.77×, 54.38×, and 12.22× faster, at only a small cost (gray regions) of 6.40 ms, 14.22 ms, 6.45 ms, and 18.23 ms, respectively. This runtime advantage is even more remarkable for large e-graphs, such as mul112 and mul96map ((e)–(f)), where XEG^{-D} cannot converge within the time limit.

Regarding the average solution quality, XEG^{-D} and XEG achieve comparable normalized costs (1.01 vs 1.01). The largest difference occurs in BoolE (1.13 vs 1.00) since its e-graphs are much larger, where the ILP solver cannot converge without DAG construction. For IMpress, Tensat, and Rover, we observe that XEG^{-D} is better than XEG (0.92 vs 1.00, 0.93 vs 1.00, and 0.98 vs 1.00, respectively). This is because our DAG construction may remove e-nodes that introduce cycles, inevitably pruning parts of the search space and preventing the ILP solver from finding better solutions. While this is a limitation, it enables the ILP solver to converge much faster (26.19× over XEG^{-D}).

Similar to tree construction, XEG's DAG construction can also be implemented on CPUs when GPUs are unavailable. To demonstrate this applicability, we implemented a CPU reference that used OpenMP `parallel-for` to parallelize our DAG construction across four operations: (1) in-degree updates (lines 4–6 in Algorithm 2), (2) frontier-based levelization across e-nodes (lines 7–14 in Algorithm 2), (3) reverting invalid insertions in parallel (lines 15–18 in Algorithm 2), and (4) e-node insertion (lines 22–33 in Algorithm 2). Atomic operations, including `atomicAdd` and `atomicSub`, are used for concurrent updates to node degrees and insertion counts (lines 6, 13, 17, and 33 in Algorithm 2). These atomic operations are necessary because node in-degrees must be updated immediately such that the latest value can correctly determine the next frontier e-nodes. As shown in Table 5, relative to the 1-thread CPU implementation, the 8-, 16-, and 32-thread CPU implementations achieve geometric-mean speedups of 1.18×, 1.23×, and 1.41×, respectively, while the GPU implementation achieves a 2.64× speedup.

Table 5. Runtime of our DAG-construction algorithm on CPUs and GPU. Runtimes are in milliseconds (ms); values in parentheses denote the percentage (%) of the total runtime. The bottom row reports the geomean speedup over the 1-thread CPU.

Benchmark	CPU				GPU
	1	8	16	32	
BoolE	368 (0.68)	238 (0.44)	191 (0.35)	159 (0.29)	69.24 (0.09)
ESyn	24.66 (0.14)	19.98 (0.12)	20.13 (0.12)	15.73 (0.09)	15.39 (0.16)
E-morphic	70.26 (0.13)	53.49 (0.10)	46.46 (0.08)	31.29 (0.06)	25.66 (0.05)
IMpress	22.08 (8.85)	16.88 (6.77)	16.59 (6.65)	16.30 (6.53)	12.14 (4.77)
Tensat	15.90 (6.40)	18.28 (7.36)	20.08 (8.09)	17.60 (7.09)	9.38 (3.53)
Flexc	48.52 (63.66)	43.27 (56.78)	42.69 (56.02)	42.28 (55.48)	13.8 (31.99)
Rover	103 (9.80)	92.8 (8.79)	88.02 (8.34)	88.57 (8.39)	33.3 (6.36)
Diospyros	17.96 (16.26)	16.46 (14.91)	13.51 (12.23)	11.44 (10.36)	4.96 (37.68)
Maxsat	0.59 (1.42)	0.77 (1.83)	1.31 (3.12)	1.42 (3.41)	1.32 (2.71)
Speedup	**1.00**	**1.18**	**1.23**	**1.41**	**2.64**

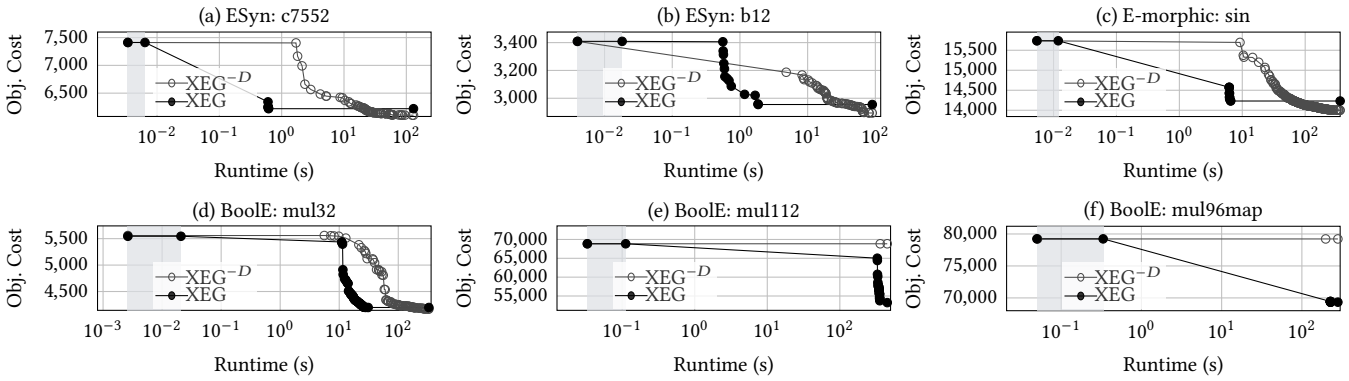


Figure 10. Convergence comparison between XEG and XEG^{-D}. The gray region denotes the runtime of DAG construction.

4.5 Scalability of DAG Construction

In this section, we evaluate the scalability of our DAG-construction kernel across five configurations and varying e-graph extraction complexity. These configurations differ in the number of threads used to handle each e-node insertion in DAG construction: one thread (Thread); 32 threads with and without the bitmap (Warp and Warp^{-b}); and 64 threads with and without the bitmap (Block and Block^{-b}). Since e-graph connectivity dominates the extraction complexity [20], we scale the number of outgoing edges of each e-node by a factor of 2. Specifically, we select two large e-graphs, mul32 and mul112, and randomly add outgoing edges from each e-node to leaf e-nodes, ensuring that no invalid or cyclic dependencies occur. We measure the effectiveness of each configuration based on its runtime relative to the Thread baseline, to determine which configuration remains most effective as the problem size grows.

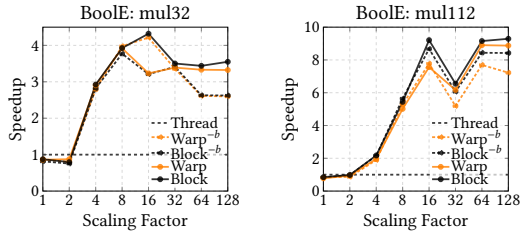


Figure 11. Scalability comparison of different DAG-construction configurations on mul32 and mul112.

As shown in Figure 11, when the e-graph is small, the Thread kernel runs fastest due to its minimal parallelization overhead. As the graph grows larger, the Thread kernel loses efficiency, while Warp and Block kernels become much faster. On mul32, when the scaling factor reaches 128, Warp^{-b}, Block^{-b}, Warp, and Block achieve speedups of 2.60×, 2.62×, 3.32×, and 3.55×, respectively, because we have more threads to detect the cycle during the e-node insertion (lines 25–33 in Algorithm 2); similar results are observed for mul112. In addition, enabling bitmap further improves the performance by reducing dependency checking from a linear scan to a constant-time lookup (line 22 in Algorithm 3). For example, on mul32 and mul112, Warp achieves additional average speedups of 1.09× and 1.22×, while Block achieves 1.03× and 1.09×, respectively, across all scaling factors compared to their bitmap-disabled variants.

4.6 Solution Quality vs Resource Constraints

As positioned in Section 3, XEG is a heuristic that focuses on exact solving within a restricted search space; the limitation is that the solution quality is constrained by the DAG construction and the ILP budget. Therefore, in this experiment, we study the potential gap to optimality by running CP-SAT in an unconstrained search space (XEG^{-D}) with a higher runtime budget of 24 hours, denoted as ILP*. To make a fair comparison, we use the same initial solution generated by XEG's tree-construction algorithm for ILP*. Under this setting, when ILP* proves optimality within 24 hours, we use the resulting optimum to measure XEG's optimality; otherwise, we report the best feasible solution found before termination, including early termination due to `OOM`.

As shown in Table 6, across all benchmarks, ILP* can further improve the solution quality by an average of 1% yet at the cost of 167× more runtime. Specifically, ILP* achieve optimal results for six benchmarks, indicating XEG is optimal for Flexc, Diospyros, and Maxsat. For ESyn, ILP* still fails to converge within 24 hours while the best solution improves about 2% over XEG. Figure 12(a) further illustrates this trade-off between runtime and solution improvement, measured at mul5 of ESyn. ILP* finds a feasible solution with a cost of 595 in approximately 35 seconds, but requires four more hours to reduce the cost to 585, an additional improvement of only 1.68%. Beyond four hours, ILP* cannot find any better feasible solution before reaching the time limit. On the other hand, ILP* improves the solution quality of BoolE and E-morphic to 0.99 before termination due to `OOM`. Figure 12(b) illustrates this memory cost for mul64 of BoolE. Doubling the memory usage improves the solution cost from 16986 to 16541 (2.62%), while another 2× increase yields only a 0.18% improvement,

from 16541 to 16511, before encountering OOM. These results demonstrate that while ILP* can provide marginally better solutions, achieving these improvements requires substantially larger runtime and memory footprint, making it less practical for large-scale e-graph applications.

Table 6. Performance comparison between XEG and ILP*. The column "OOM" reports the number of e-graphs where ILP* terminates due to out-of-memory. The column "Optimal" indicates whether ILP* reaches the optimum for all e-graphs in each benchmark. For runs that do not reach optimality, we report the best feasible solution found before termination and their OOM counts. Detailed per-e-graph results are available in Appendix B.

Benchmark	Avg. runtime ratio		Avg. normalized cost			ILP*	
	ILP*	XEG	ILP*	XEG	Optimal	OOM	Optimal
BoolE	19.61	1.00	0.99	1.00	N/A	6	×
ESyn	958	1.00	0.98	1.00	N/A	0	×
E-morphic	8.88	1.00	0.99	1.00	N/A	1	×
IMpress	16.11	1.00	0.92	1.00	×	0	✓
Tensat	19.33	1.00	0.93	1.00	×	0	✓
Flexc	14.09	1.00	1.00	1.00	✓	0	✓
Rover	20.21	1.00	0.98	1.00	×	0	✓
Diospyros	17.90	1.00	1.00	1.00	✓	0	✓
Maxsat	0.98	1.00	1.00	1.00	✓	0	✓
Overall Avg.	**167**	**1.00**	**0.99**	**1.00**	–	–	–

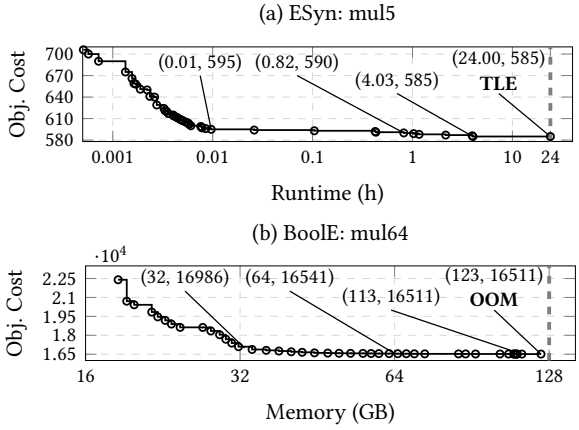


Figure 12. Solution-quality tradeoff of ILP under resource constraints. ILP* offers only marginal gains at substantial runtime and memory costs.

5 Conclusion

In this paper, we have presented XEG, a GPU-parallel e-graph extractor that can achieve both high runtime efficiency and solution quality. Experimental results show that XEG achieves a 243× average speedup over SmoothE while improving solution quality by at least 7%. Compared with e-boost, XEG achieves a 7.73× average speedup while maintaining comparable solution quality. In addition, XEG can operate without ILP as a standalone extractor, achieving a 123× average speedup over egg with comparable solution quality.

Acknowledgments

This work was jointly supported by the National Science Foundation (NSF) under Grant No. 2235276, 2349144, 2349143, 2349582, and 2349141.

A Results with a GPU-based ILP Solver

To study the impact of the ILP solver, we replace CP-SAT [29] in XEG with NVIDIA cuOpt [39], a GPU-accelerated ILP solver, denoted as XEG_{cuOpt}. For a fair comparison, XEG_{cuOpt} uses the solution produced by our GPU tree construction as a warm start for cuOpt. The cuOpt timeout is set to 360 seconds, the same as that of CP-SAT used in XEG.

Table 7. Performance comparison between CPU- and GPU-based ILP solver used in XEG. XEG_{cuOpt} replaces the CPU-based CP-SAT [29] solver with the state-of-the-art GPU-based ILP solver, NVIDIA cuOpt [39].

Benchmark	Avg. runtime ratio		Avg. normalized cost	
	XEG_{cuOpt}	XEG	XEG_{cuOpt}	XEG
BoolE	46.85	1.00	1.21	1.00
ESyn	177	1.00	1.09	1.00
E-morphic	15.76	1.00	1.13	1.00
IMpress	1.07	1.00	1.00	1.00
Tensat	12.55	1.00	1.00	1.00
Flexc	11.41	1.00	1.00	1.00
Rover	92.56	1.00	1.00	1.00
Diospyros	13.21	1.00	1.00	1.00
Maxsat	22.32	1.00	1.00	1.00
Overall Avg.	**54.12**	**1.00**	**1.06**	**1.00**

As shown in Table 7, XEG_{cuOpt} is 54.12× slower than XEG on average and produces 6% worse solution quality across all evaluated e-graphs. We observed that XEG_{cuOpt} does not reach the same solution quality as XEG on BoolE, ESyn, and E-morphic within the time limit. We believe this is because the current cuOpt employs a hybrid CPU/GPU approach, where GPU-accelerated primal heuristics are used to improve the primal bound, while branch-and-bound search is used to improve the dual bound [39]. Moreover, the current cuOpt solver does not support using a warm start together with presolve [38] like CP-SAT, which can further limit its performance. To sum up, although GPU is effective for parallelizing our tree and DAG constructions, it does not provide the same benefit for the subsequent ILP solving. We thereby use CP-SAT as the ILP solver in XEG.

B Detailed Per-E-Graph Statistics and Performance Comparison

Table 8 presents detailed e-graph statistics and a performance comparison of XEG with three state-of-the-art e-graph extractors: egg [55], SmoothE [2], and e-boost [59], as well as ILP*.

Table 8. Performance comparison of XEG with state-of-the-art e-graph extractors, egg [55], SmoothE [2], and e-boost [59], as well as the ILP*. † denotes the best feasible solution found by ILP* before termination, including early termination due to OOM.

Benchmark	e-graph	#e-classes	#e-nodes	$\bar{d}^{NC}$	$\bar{d}^{NN}$	Runtime (ms)					Final objective cost				
						egg	SmoothE	e-boost	ILP*	XEG	egg	SmoothE	e-boost	ILP*	XEG
BoolE [60]: Boolean algebra and logic synthesis	mul16	4226	10509	1.82	4.85	23.71	46789	39470	93859	1815	1354	1160	1054	1054	1057
	mul32	17666	44557	1.82	4.96	190	481197	339340	2061621	26993	5540	5043	4162	4158	4167
	mul48	40322	102157	1.81	4.99	701	1027247	361422	13706640	311197	12557	12139	9937	9310	9437
	mul64	72194	183309	1.82	5.01	1651	1201320	359241	13816053†	359240	22381	23862	19548	16512†	16755
	mul80	113282	288013	1.82	5.02	3310	OOM	374508	5047064†	218506	35062	OOM	36332	25790†	25889
	mul96	163586	416269	1.82	5.03	4727	OOM	383647	3267086†	359877	50540	OOM	52476	39293†	37199
	mul112	223106	568077	1.82	5.04	8215	OOM	402540	2245112†	340472	68821	OOM	71336	57712†	53745
	mul128	291842	743437	1.82	5.04	10950	OOM	OOM	1899312†	360540	89930	OOM	OOM	88569†	81902
	mul16map	8695	17201	1.80	3.55	56.59	85978	3866	4391	782	1882	1662	1612	1612	1653
	mul32map	37357	73442	1.80	3.53	752	520798	36955	41172	7910	8492	7376	7030	7030	7259
	mul48map	86147	168940	1.80	3.51	3407	1203462	146936	167664	28833	20063	27279	16317	16317	16885
	mul64map	154812	303327	1.80	3.50	8316	OOM	358203	409220	73246	36392	OOM	29440	29440	30502
	mul80map	242390	474818	1.80	3.51	29668	OOM	439548	823011	133367	57397	OOM	51563	46238	48023
	mul96map	348800	683229	1.80	3.51	47853	OOM	OOM	1786514	228228	82770	OOM	OOM	66683	69333
	mul112map	474045	928622	1.80	3.51	88182	OOM	OOM	2858725	358067	112813	OOM	OOM	90810	93567
	mul128map	618816	1212206	1.80	3.51	131399	OOM	OOM	4555545†	360788	147391	OOM	OOM	129327†	140830
Avg. runtime ratio / normalized cost						**0.097**	**≥38.28**	**≥5.28**	**19.61**	**1.00**	**1.23**	**≥1.22**	**≥1.12**	**0.99**	**1.00**
ESyn [7]: Logic synthesis	mul3	3360	8544	1.78	16.73	11.48	18974	25990	500571	315	201	171	151	150	160
	c7552	5668	9988	1.79	3.40	8.70	35689	7158	165519	629	7412	6464	6166	6102	6220
	b12	5833	10987	1.80	7.22	9.72	31703	127342	74401	2039	3176	3084	2919	2894	2955
	i7	5468	11241	1.73	13.28	9.19	16554	34752	179517	24533	2046	2008	1925	1897	1909
	C432	5144	11695	1.76	19.14	14.88	17882	192373	76931	5194	598	620	550	547	553
	c2670	6532	13205	1.75	20.62	17.23	34461	297743	496026	12937	2423	2420	2174	2170	2201
	C5315	8000	13774	1.70	7.66	15.82	12690	81861	125665	6671	5466	5180	4750	4728	4837
	mul5	5941	13967	1.74	10.30	24.27	15218	290123	86401017†	62915	706	741	589	585†	599
	qdiv	7872	14604	1.77	3.86	15.59	27521	11760	100377	1074	6058	6013	5370	5370	5381
	max	11578	17503	1.80	2.59	117	162649	221088	86402021†	14967	10105	10443	9017	9015†	9160
	bar	12045	19068	1.68	2.68	19.12	13626	26517	22386	9597	9387	9410	9113	9113	9113
	cavlc	10649	23728	1.76	5.68	29.60	22885	337213	23374218	287030	2099	2158	1890	1835	1886
	adder	15102	29701	1.89	4.95	31.25	170090	131073	86400030†	21045	3823	3829	3445	3445†	3447
	frg2	14640	32022	1.74	6.66	39.52	27453	351510	3555411	212891	3190	2950	2796	2764	2808
Avg. runtime ratio / normalized cost						**0.006**	**13.40**	**19.44**	**958**	**1.00**	**1.12**	**1.09**	**0.99**	**0.98**	**1.00**
E-morphic [6]: Compiler optimization	sin	13399	20390	1.78	2.94	139	163012	318652	219731	6614	15759	15343	14048	13846	14231
	arbiter	29993	36257	1.60	2.60	29.17	71285	9518	51245	11871	35875	36196	35583	35455	35585
	square	60333	88865	1.74	2.40	406	51549	51565	56197	52167	56014	54801	53391	53391	53653
	multiplier	72489	94459	1.64	2.17	184	119555	74068	62422	60125	77990	78177	77821	77821	77821
	log2	98112	142619	1.59	2.67	4784	625277	374246	7051072†	399274	93533	87632	84579	84181†	84211
	mem_ctrl	124478	165084	1.72	2.22	421	107506	52637	50799	49308	133470	136175	132986	132982	132982
	div	146160	190310	1.66	2.49	391	1204408	452381	1386026	360671	159362	159570	161006	159328	159358
Avg. runtime ratio / normalized cost						**0.008**	**5.82**	**7.78**	**8.88**	**1.00**	**1.04**	**1.03**	**1.00**	**0.99**	**1.00**
IMpress [51]: FPGA HLS	mul512	9127	10275	2.00	2.11	7.45	25731	356	615	83.60	149213	136680	136680	136680	149213
	mul1024	46180	51648	2.00	2.11	56.45	33680	2612	4653	382	495629	427721	424418	424418	495629
	mul2048	90312	102030	2.00	2.15	123	38478	6981	14797	514	3671672	2062981	1305965	1305960	1305960
Avg. runtime ratio / normalized cost						**0.159**	**157**	**8.23**	**16.11**	**1.00**	**1.60**	**1.12**	**0.92**	**0.92**	**1.00**
Tensat [58]: Tensor graph [18, 23, 43])	vgg	1408	2726	2.70	5.98	1.75	43637	111	225	26.17	468	470	468	468	468
	resnet50	2656	4350	2.75	2.28	2.03	83848	264	380	57.32	398	404	398	397	398
	bert	7315	20244	2.03	11.10	13.62	23723	967	22261	670	64	62	64	48	57
	nasneta	28000	43072	1.83	4.49	258	43051	5065	101968	3714	1728	1082	1154	994	1208
	nasrnn	34800	57800	2.62	3.10	79.43	OOM	3621	7347	354	35	OOM	35	35	35
Avg. runtime ratio / normalized cost						**0.083**	**≥794**	**4.38**	**19.33**	**1.00**	**1.11**	**≥1.00**	**1.02**	**0.93**	**1.00**
Flexc [56]: CGRA mapping (Bzip2 [42] and FFmpeg [50])	flexc_75	1734	6768	1.88	40.20	4.84	8506	153	386	30.68	35	35	35	35	35
	flexc_67	1734	6768	1.88	40.20	5.03	7615	151	391	31.38	37	37	36	36	36
	flexc_db	1734	6768	1.86	40.20	4.73	8938	128	370	31.22	37	37	36	36	36
	flexc_86	1734	6768	1.88	40.20	4.85	8918	127	374	31.15	37	37	36	36	36
	flexc_4a	1734	6768	1.88	40.20	4.70	8895	127	378	31.34	37	37	36	36	36
	flexc_03	1734	6768	1.87	40.20	4.89	8870	126	376	31.20	37	37	36	36	36
	flexc_7b	2921	11779	1.86	60.22	8.57	12381	236	723	47.93	95	96	95	95	95
	flexc_ff	3276	12423	1.88	55.67	9.53	21894	285	723	57.21	134	135	134	134	134
	flexc_b8	3276	12423	1.86	55.67	9.37	38996	180	757	58.68	136	141	136	136	136
	flexc_a0	3276	12423	1.86	55.67	9.30	38495	182	743	58.47	136	141	136	136	136
	flexc_85	3276	12423	1.88	55.67	9.28	37950	185	771	58.54	136	142	136	136	136
	flexc_5c	3276	12423	1.89	55.67	9.35	39392	183	754	58.43	136	141	136	136	136
	flexc_fd	2499	12458	1.86	63.10	9.23	12998	363	779	52.56	105	106	105	105	105
	flexc_fe	4892	19830	1.86	55.76	17.01	20438	402	1504	50.04	88	88	88	88	88
Avg. runtime ratio / normalized cost						**0.174**	**400**	**4.53**	**14.09**	**1.00**	**1.01**	**1.02**	**1.00**	**1.00**	**1.00**
Rover [15]: Datapath synthesis (Mediabench [30] and MCM [22])	box5r	349	1838	9.39	60.11	2.05	3947	69.01	117	84.04	1819	1882	1819	1819	1819
	box3r	666	2369	9.92	25.20	2.99	18748	427	776	605	1701	1760	1701	1701	1701
	fir8t5r	1604	5289	10.15	35.01	7.88	22326	1361	550	401	5936	6307	5936	5936	5936
	fir8t6r	1022	8837	8.30	55.41	13.62	24983	998	1392	608	5936	6873	5936	5936	5936
	fir8t7r	849	12224	8.60	76.75	24.27	25192	1959	32476	581	5936	7332	5936	5936	5936
	box4r	2852	12537	9.52	84.09	18.23	58208	4473	94204	865	1762	1761	1659	1635	1635
	fir8t8r	723	13037	9.57	87.60	25.84	15112	1521	1788	622	5936	6832	5936	5936	5936
	mcm378r	2369	16206	8.45	678	23.51	29219	896	3338	936	1165	1095	1165	1050	1165
	mcm379r	2694	16960	8.38	663	27.80	31037	977	3495	818	1165	1982	1165	1050	1165
Avg. runtime ratio / normalized cost						**0.026**	**38.63**	**2.19**	**20.21**	**1.00**	**1.01**	**1.15**	**1.00**	**0.98**	**1.00**
Diospyros [53]: DSP vectorization	vaddr7	17	91	2.73	6.38	0.02	695	5.75	4.87	3.22	1	1	1	1	1
	vmjmzr14	46	369	2.81	11.67	0.09	1269	14.33	7.78	4.01	1	1	1	1	1
	vvamr23	78	525	1.26	9.73	0.11	1753	14.24	9.65	3.77	1	1	1	1	1
	vpmr23	78	525	2.26	9.73	0.10	1857	15.79	9.69	3.75	1	1	1	1	1
	vmacr21	135	2312	2.17	20.70	0.61	6620	63.01	45.02	4.23	1	1	1	1	1
	vmm22r21	215	3142	2.23	20.41	0.74	4995	49.62	59.16	5.05	1	1	1	1	1
	vmm22r26	219	3172	1.86	20.37	1.33	5286	59.23	61.45	6.24	1	1	1	1	1
	specr26	219	3172	2.38	20.37	1.33	5005	59.27	61.43	6.28	1	1	1	1	1
	vcv22r36	297	3764	2.26	17.80	0.88	7317	83.60	71.65	5.83	1	1	1	1	1
	vmm23r58	1086	15384	2.26	21.07	7.57	15311	424	319	18.48	1	1	1	1	1
	vcv33r168	9584	144861	2.21	20.90	60.64	OOM	1349	4375	105	1	OOM	1	1	1
	drcr2r18	643	154773	1.63	301	161	OOM	1467	24823	293	1	OOM	1	1	1
	drcr18	643	218933	1.63	380	255	OOM	2154	18416	703	1	OOM	1	1	1
Avg. runtime ratio / normalized cost						**0.219**	**≥777**	**8.86**	**17.90**	**1.00**	**1.00**	**≥1.00**	**1.00**	**1.00**	**1.00**
Maxsat [2]: Maximum satisfiability [1]	maxcut39	1401	1541	1.95	1.82	0.65	731	34.00	31.28	29.63	141	141	141	141	141
	maxcut1	1401	1541	1.94	1.82	0.65	735	35.30	31.47	30.26	141	141	141	141	141
	s2v1	2741	2881	1.73	1.90	1.43	762	59.59	57.57	54.15	141	141	141	141	141
	s2v3	2741	2881	1.73	1.90	1.46	801	59.09	57.58	54.08	141	141	141	141	141
	hamming6	3713	3777	1.85	1.97	1.95	1265	79.90	67.23	72.06	65	65	65	65	65
	johnson8	3781	3851	1.85	1.96	2.01	1029	80.33	69.28	71.64	71	71	71	71	71
Avg. runtime ratio / normalized cost						**0.025**	**18.29**	**1.12**	**1.02**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**
Overall Avg. runtime ratio / normalized cost						**0.095**	**≥243**	**≥7.73**	**167**	**1.00**	**1.09**	**≥1.07**	**≥1.01**	**0.99**	**1.00**

References

[1] Josep Argelich, Chu-Min Li, Felip Manya, and Jordi Planes. 2008. The first and second Max-SAT evaluations. *Journal on Satisfiability, Boolean Modelling and Computation* 4, 2-4 (2008), 251–278.

[2] Yaohui Cai, Kaixin Yang, Chenhui Deng, Cunxi Yu, and Zhiru Zhang. 2025. SmoothE: Differentiable E-Graph Extraction. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1020–1034. doi:10.1145/3669940.3707262

[3] David Cao, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. 2023. babble: Learning better abstractions with e-graphs and anti-unification. *Proceedings of the ACM on Programming Languages* 7, POPL (2023), 396–424.

[4] Che Chang, Yi-Hua Chung, Cheng-Hsiang Chiu, Wan-Luan Lee, Boyang Zhang, Ulf Schlichtmann, Ing-Chao Lin, Xiangyao Yu, and Tsung-Wei Huang. 2026. G-PathGen: An Efficient GPU-Parallel k-Critical Path Generation Algorithm. In *ACM International Conference on Supercomputing (ICS)*.

[5] Chih-Chun Chang and Tsung-Wei Huang. 2025. Statistical Timing Graph Scheduling Algorithm for GPU Computation. In *ACM/IEEE Design Automation Conference (DAC)*.

[6] Chen Chen, Guangyu HU, Cunxi Yu, Yuzhe Ma, and Hongce Zhang. 2025. E-morphic: Scalable Equality Saturation for Structural Exploration in Logic Synthesis. arXiv:2504.11574 [cs.AR] https://arxiv.org/abs/2504.11574

[7] Chen Chen, Guangyu Hu, Dongsheng Zuo, Cunxi Yu, Yuzhe Ma, and Hongce Zhang. 2024. E-Syn: E-Graph Rewriting with Technology-Aware Cost Functions for Logic Synthesis. In *Proceedings of the 61st ACM/IEEE Design Automation Conference* (San Francisco, CA, USA) *(DAC '24)*. Association for Computing Machinery, New York, NY, USA, Article 124, 6 pages. doi:10.1145/3649329.3656246

[8] Hongzheng Chen, Niansong Zhang, Shaojie Xiang, Zhichen Zeng, Mengjia Dai, and Zhiru Zhang. 2024. Allo: A programming model for composable accelerator design. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 593–620.

[9] Jianyi Cheng, Samuel Coward, Lorenzo Chelini, Rafael Barbalho, and Theo Drane. 2024. Seer: Super-optimization explorer for high-level synthesis using e-graph rewriting. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1029–1044.

[10] Yi-Hua Chung, Shui Jiang, Wan Luan Lee, Yanqing Zhang, Haoxing Ren, Tsung-Yi Ho, and Tsung-Wei Huang. 2025. SimPart: A Simple Yet Effective Replication-aided Partitioning Algorithm for Logic Simulation on GPU. In *International European Conference on Parallel and Distributed Computing (Euro-Par)*.

[11] Yi-Hua Chung, Nahmsuk Oh, MG Balabhadra Naga Venkata, Aditya Shiledar, Sudipto Kundu, Vishal Khandelwal, and Tsung-Wei Huang. 2025. Accelerating gate sizing using gpu. In *Proceedings of the European Conference on Parallel Processing*.

[12] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. 2022. *Introduction to algorithms*. MIT press.

[13] Samuel Coward, George A Constantinides, and Theo Drane. 2022. Automatic datapath optimization using e-graphs. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*. IEEE, 43–50.

[14] Samuel Coward, George A Constantinides, and Theo Drane. 2023. Automating constraint-aware datapath optimization using e-graphs. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.

[15] Samuel Coward, Theo Drane, Emiliano Morini, and George A Constantinides. 2024. Combining power and arithmetic optimization via datapath rewriting. In *2024 IEEE 31st Symposium on Computer Arithmetic (ARITH)*. IEEE, 24–31.

[16] Nachum Dershowitz. 2005. A taste of rewrite systems. In *Functional Programming, Concurrency, Simulation and Automated Reasoning: International Lecture Series 1991–1992 McMaster University, Hamilton, Ontario, Canada*. Springer, 199–228.

[17] Nachum Dershowitz and Jean-Pierre Jouannaud. 1990. Rewrite systems. In *Formal models and semantics*. Elsevier, 243–320.

[18] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[19] Oliver Flatt, Samuel Coward, Max Willsey, Zachary Tatlock, and Pavel Panchekha. 2022. Small proofs from congruence closure. In *2022 Formal Methods in Computer-Aided Design (FMCAD)*. IEEE, 75–83.

[20] Amir Kafshdar Goharshady, Chun Kit Lam, and Lionel Parreaux. 2024. Fast and optimal extraction for sparse equality graphs. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 2551–2577.

[21] LLC Gurobi Optimization. 2025. The leader in decision intelligence technology–gurobi optimization. https://www.gurobi.com/

[22] Oscar Gustafsson. 2007. A difference based adder graph heuristic for multiple constant multiplication problems. In *2007 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1097–1100.

[23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[24] Mike He, Haichen Dong, Sharad Malik, and Aarti Gupta. 2023. Improving term extraction with acyclic constraints. In *EGRAPHS 2023 workshop*.

[25] Guyue Huang, Guohao Dai, Yu Wang, and Huazhong Yang. 2020. Gespmm: General-purpose sparse matrix-matrix multiplication on gpus for graph neural networks. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–12.

[26] Tsung-Wei Huang, Dian-Lun Lin, Chun-Xun Lin, and Yibo Lin. 2022. Taskflow: A Lightweight Parallel and Heterogeneous Task Graph Computing System. In *IEEE Transactions on Parallel and Distributed Systems (TPDS)*. 1303–1320.

[27] IBM. 2025. Ibm ilog cplex optimization studio. https://www.ibm.com/products/ilog-cplex-optimization-studio

[28] Arthur B Kahn. 1962. Topological sorting of large networks. *Commun. ACM* 5, 11 (1962), 558–562.

[29] Dominik Krupke, Leon Lan, Michael Perk, , et al. [n. d.]. *The CP-SAT Primer: Using and Understanding Google OR-Tools' CP-SAT Solver*. https://github.com/d-krupke/cpsat-primer

[30] Chunho Lee, Miodrag Potkonjak, and William H Mangione-Smith. 1997. Mediabench: A tool for evaluating and synthesizing multimedia and communications systems. In *Proceedings of 30th Annual International Symposium on Microarchitecture*. IEEE, 330–335.

[31] Wan-Luan Lee, Shui Jiang, Dian-Lun Lin, Che Chang, Boyang Zhang, Yi-Hua Chung, Ulf Schlichtmann, Tsung-Yi Ho, and Tsung-Wei Huang. 2025. iG-kway: Incremental k-way Graph Partitioning on GPU. In *ACM/IEEE Design Automation Conference (DAC)*.

[32] Rui Li, Lin Li, Heng Yu, and Yajun Ha. 2025. ESACO: Fast E-Graph Extraction via Orchestrated Simulated Annealing-based Local Search and Ant Colony Optimization-based Global Search. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).

[33] Kazuaki Matsumura, Simon Garcia De Gonzalo, and Antonio J Peña. 2024. ACC Saturator: Automatic Kernel Optimization for Directive-Based GPU Code. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1979–1990.

[34] Duane Merrill, Michael Garland, and Andrew Grimshaw. 2012. Scalable GPU graph traversal. *ACM Sigplan Notices* 47, 8 (2012), 117–128.

[35] Greg Nelson and Derek C Oppen. 1979. Simplification by cooperating decision procedures. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 1, 2 (1979), 245–257.

[36] Greg Nelson and Derek C Oppen. 1980. Fast decision procedures based on congruence closure. *Journal of the ACM (JACM)* 27, 2 (1980), 356–364.

[37] Julie L Newcomb, Andrew Adams, Steven Johnson, Rastislav Bodik, and Shoaib Kamil. 2020. Verifying and improving halide's term rewriting system with program synthesis. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–28.

[38] NVIDIA. 2026. cuOpt MIP C API Reference. https://docs.nvidia.com/cuopt/user-guide/latest/cuopt-c/mip/mip-c-api.html.

[39] NVIDIA. 2026. NVIDIA cuOpt User Guide. https://docs.nvidia.com/cuopt/user-guide/latest/.

[40] Pavel Panchekha, Alex Sanchez-Stern, James R Wilcox, and Zachary Tatlock. 2015. Automatically improving accuracy for floating point expressions. *Acm Sigplan Notices* 50, 6 (2015), 1–11.

[41] Rinus Plasmeijer, Marko Van Eekelen, and MJ Plasmeijer. 1993. *Functional programming and parallel graph rewriting*. Vol. 857. Addison-wesley Reading.

[42] Julian Seward. 1996. bzip2 and libbzip2. *available at http://www. bzip. org* (1996), 8–18.

[43] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).

[44] Zak Singh. 2022. Deep reinforcement learning for equality saturation. *MPhil Research Project. University of Cambridge, June* (2022).

[45] Gus Henry Smith, Andrew Liu, Steven Lyubomirsky, Scott Davidson, Joseph McMahan, Michael Taylor, Luis Ceze, and Zachary Tatlock. 2021. Pure tensor program rewriting via access patterns (representation pearl). In *Proceedings of the 5th ACM SIGPLAN International Symposium on Machine Programming*. 21–31.

[46] Gus Henry Smith, Zachary D Sisco, Thanawat Techaumnuaiwit, Jingtao Xia, Vishal Canumalla, Andrew Cheung, Zachary Tatlock, Chandrakana Nandi, and Jonathan Balkind. 2024. There and back again: A netlist's tale with much egraphin'. *arXiv preprint arXiv:2404.00786* (2024).

[47] Michael Benjamin Stepp. 2011. *Equality saturation: engineering challenges and applications*. University of California, San Diego.

[48] Robert Endre Tarjan. 1975. Efficiency of a good but not linear set union algorithm. *Journal of the ACM (JACM)* 22, 2 (1975), 215–225.

[49] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. Equality saturation: a new approach to optimization. In *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 264–276.

[50] Suramya Tomar. 2006. Converting video formats with FFmpeg. *Linux journal* 2006, 146 (2006), 10.

[51] Ecenur Ustun, Ismail San, Jiaqi Yin, Cunxi Yu, and Zhiru Zhang. 2022. IMpress: Large Integer Multiplication Expression Rewriting for FPGA HLS. In *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 1–10.

[52] Mark GJ Van den Brand, Jan Heering, Paul Klint, and Pieter A Olivier. 2002. Compiling language definitions: the ASF+ SDF compiler. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 24, 4 (2002), 334–368.

[53] Alexa VanHattum, Rachit Nigam, Vincent T Lee, James Bornholt, and Adrian Sampson. 2021. Vectorization for digital signal processors via equality saturation. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 874–886.

[54] Yisu Remy Wang, Shana Hutchison, Jonathan Leang, Bill Howe, and Dan Suciu. 2020. SPORES: sum-product optimization via relational equality saturation for large scale linear algebra. *arXiv preprint arXiv:2002.07951* (2020).

[55] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. Egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 1–29.

[56] Jackson Woodruff, Thomas Koehler, Alexander Brauckmann, Chris Cummins, Sam Ainsworth, and Michael FP O'Boyle. 2023. Rewriting History: Repurposing Domain-Specific CGRAs. *arXiv preprint arXiv:2309.09112* (2023).

[57] Chenming Wu, Haisen Zhao, Chandrakana Nandi, Jeffrey I Lipton, Zachary Tatlock, and Adriana Schulz. 2019. Carpentry compiler. *ACM Transactions on Graphics (TOG)* 38, 6 (2019), 1–14.

[58] Yichen Yang, Phitchaya Phothilimthana, Yisu Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. 2021. Equality saturation for tensor graph superoptimization. *Proceedings of Machine Learning and Systems* 3 (2021), 255–268.

[59] Jiaqi Yin, Zhan Song, Chen Chen, Yaohui Cai, Zhiru Zhang, and Cunxi Yu. 2025. e-boost: Boosted E-Graph Extraction with Adaptive Heuristics and Exact Solving. arXiv:2508.13020 [cs.AI] https://arxiv.org/abs/2508.13020

[60] Jiaqi Yin, Zhan Song, Chen Chen, Qihao Hu, and Cunxi Yu. 2025. BoolE: Exact Symbolic Reasoning via Boolean Equality Saturation. arXiv:2504.05577 [cs.AR] https://arxiv.org/abs/2504.05577

[61] Yihong Zhang. 2023. The E-graph extraction problem is NP-complete. https://effect.systems/blog/egraph-extraction.html

[62] Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. 2023. Better together: Unifying datalog and equality saturation. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 468–492.